JavaFX Technology

Building GUI Applications With JavaFX - Tutorial Overview

This tutorial presents basic concepts for creating graphical user interfaces, including declarative syntax, nodes, shapes, visual effects, animation, layout, and event handling. Before starting this tutorial, learn about core concepts and language syntax from the Learning the JavaFX Script Programming Language.

Note: For instructions on downloading and installing the necessary software, see Getting Started With JavaFX Script of the Language Tutorial.

The lessons in this tutorial include:

- Lesson 1: Quick JavaFX GUI Overview — A visual guide to basic features available through the JavaFX API. The screen shots in this lesson display graphical objects, UI components, effects, text patterns, color schemes, and layout patterns.
- Lesson 2: Using Declarative Syntax — An introduction to the declarative syntax of JavaFX Script programming language. A step-by-step procedure describes how to create a simple GUI application.
- Lesson 3: Presenting UI Objects in a Graphical Scene — A description of basic concepts in the node architecture and the scene graph that underly the JavaFX Script programming language. You will build a graphical scene of an application, create a group of nodes, and apply a transformation to the group.
- Lesson 4: Creating Graphical Objects — An introduction to creating sophisticated graphical objects. You will create a record button for an audio player with a reflection effect.
- Lesson 5: Applying Data Binding to UI Objects — A description of the data binding mechanism with a practical example.
- Lesson 6: Laying Out GUI Elements — An explanation of how to layout UI elements in JavaFX applications with an example that illustrates the approach and techniques.
- Lesson 7: Creating Animated Objects — An explanation of how to build a graphical object and then animate it using linear interpolation, a type of key frame animation supported by JavaFX libraries.
- Lesson 8: Bringing Interactivity to GUI Elements — A description of how to create interactive applications. A step-by step procedure shows how to add behavior to a button application via handling mouse events.

Tutorial Contents

http://java.sun.com/javafx/1/tutorials/ui/overview/ Feb 10, 2009

Building GUI Applications With JavaFX

Lesson 1: Quick JavaFX GUI Overview

Download tutorial

This lesson introduces visual guides to the basic features available through the JavaFX API. It contains screen captures of graphical objects, components, effects, text patterns, color schemes, and layout patterns. Source files are provided for all visual guides.

The JavaFX API enables developers to create UIs that work seamlessly across different devices. The *common* profile of the JavaFX API includes classes that function on both the desktop and mobile devices. However, you can use additional classes and packages from the *desktop* profile to take advantage of specific functionality that can enhance desktop applications.

- Common Profile
- Desktop Profile

The JavaFX SDK contains the JavaFX Mobile Emulator, a mobile phone simulation. Use the emulator to see how your applications will look on mobile devices. Refer to the SDK Readme file (<*SDK-install-directory*>/README.html) for more information on the mobile emulator. The following images show how a simple JavaFX application will run on the emulator and in the desktop window. The complete code of this application is available in the `ball.fx` file.

Figure 1: A Simple Graphical Application Run on the Mobile Emulator

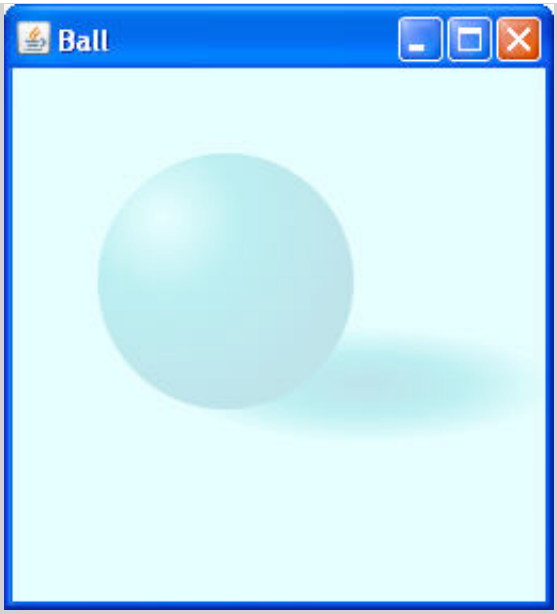

Figure 2: A Simple Graphical Application Run in the Desktop Window

Common Profile

- Colors
- Shapes
- Fill Styles
- Line Cap and Join Style
- Text
- Transformations
- Layout

Colors

The following window is displayed when you run the application code provided in the **`colors.fx`** file. This `colors` application illustrates the color patterns for all constants of the `javafx.scene.paint.Color` class. Click a color pattern to fill the scene with the corresponding color, such as `Color.FORESTGREEN`, `Color.YELLOW`, and `Color.YELLOWGREEN`.

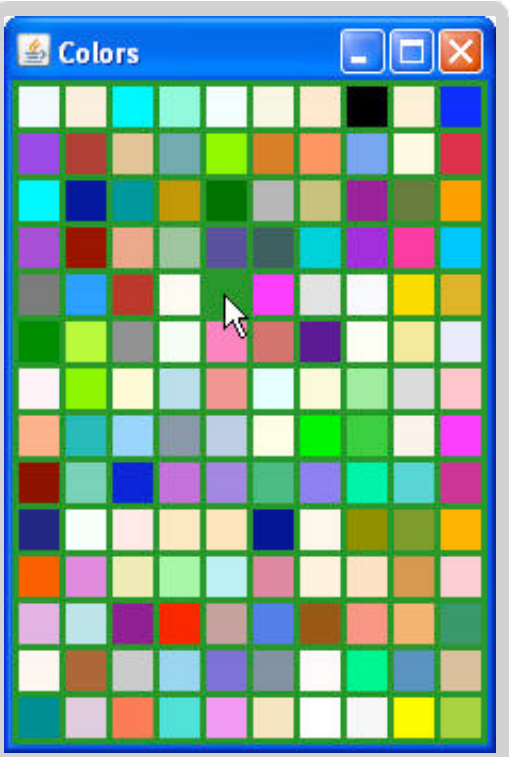

Figure 3: Color Patterns

Color schemes are employed in the following lessons: Presenting UI Objects in a Graphical Scene, Creating Graphical Objects, Applying Data Binding to UI Objects, Laying Out GUI Elements, Creating Animated Objects, and Bringing Interactivity to GUI Elements.

Shapes

This screen capture shows basic geometric primitives and shapes you can create using the `javafx.scene.shape` package.

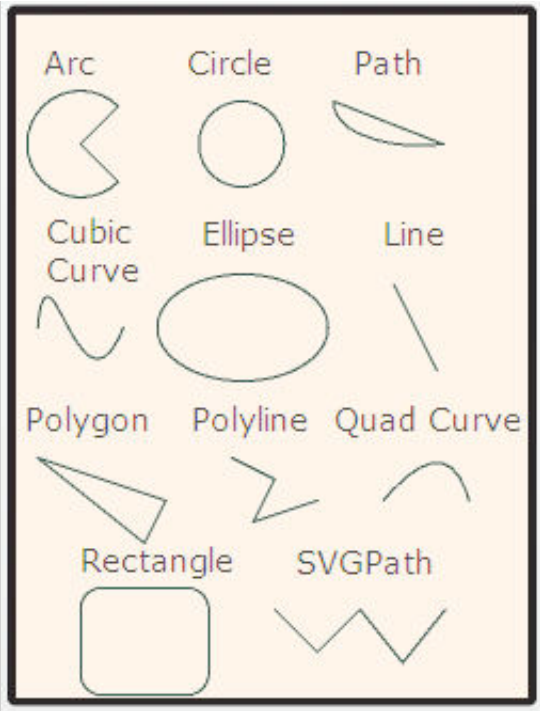

Figure 4: Basic shapes

Find the complete code of this application in the `shapes.fx` file. Note that the text captions on the screen capture are not part of the example code.

Fill Styles

This application illustrates the basic fill methods available in the `javafx.scene.paint` package. You can create various filling patterns for the scene and shapes.

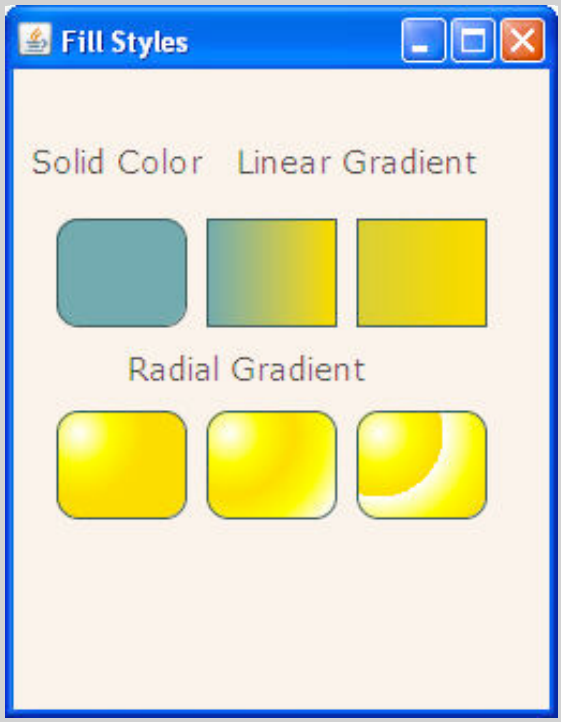

Figure 5: Basic filling methods

Find the complete code of this application in the **`fill.fx`** file. Note that the text captions on the screen capture are not part of the example code. Geometric shapes and different fill styles are discussed in the following lessons: Using Declarative Syntax, Presenting UI Objects in a Graphical Scene, Creating Graphical Objects, Applying Data Binding to UI Objects, Laying Out GUI Elements, Creating Animated Objects, and Bringing Interactivity to GUI Elements.

Line Cap and Join Styles

When constructing geometric figures you can use different methods to join and end subpaths. The following screen capture shows the basic caps and joins available in the `javafx.scene.shape` package.

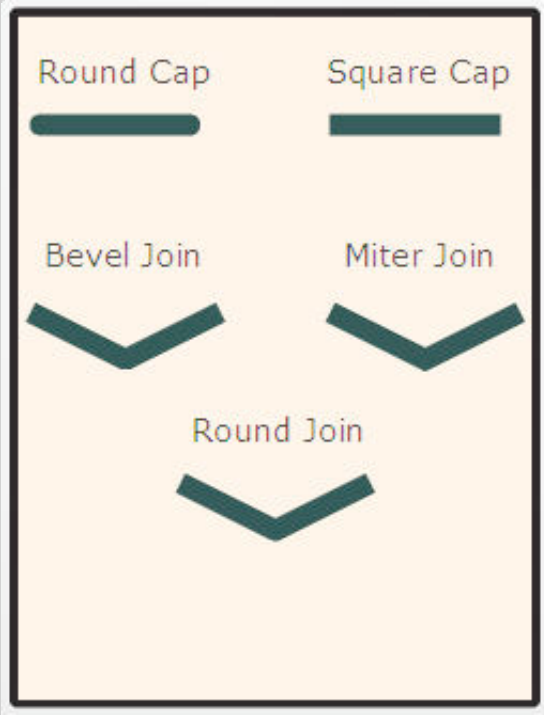

Figure 6: Line Cap and Join Styles

Find the complete code of this application in the **`line.fx`** file. Note that the text captions on the screen capture are not part of the example code.

Text

The following window is displayed when you run the application code provided in the **`text.fx`** file. This `text` application displays samples of different formatting styles applied to the same text string.

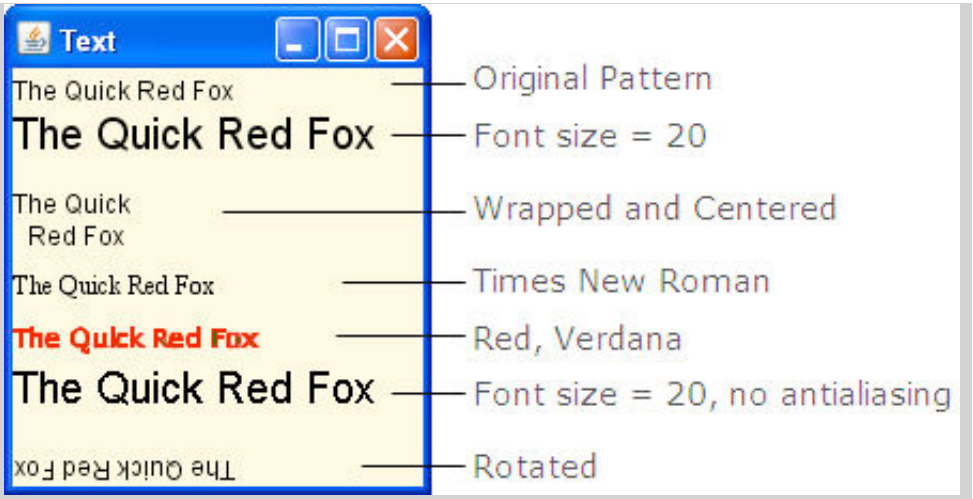

Figure 7: Text samples

Using text components is discussed in the following lessons: Presenting UI Objects in a Graphical Scene and Laying Out GUI Elements.

Transformations

The following screen capture demonstrates the basic transformations that can be performed for the graphics, images, or text in JavaFX applications.

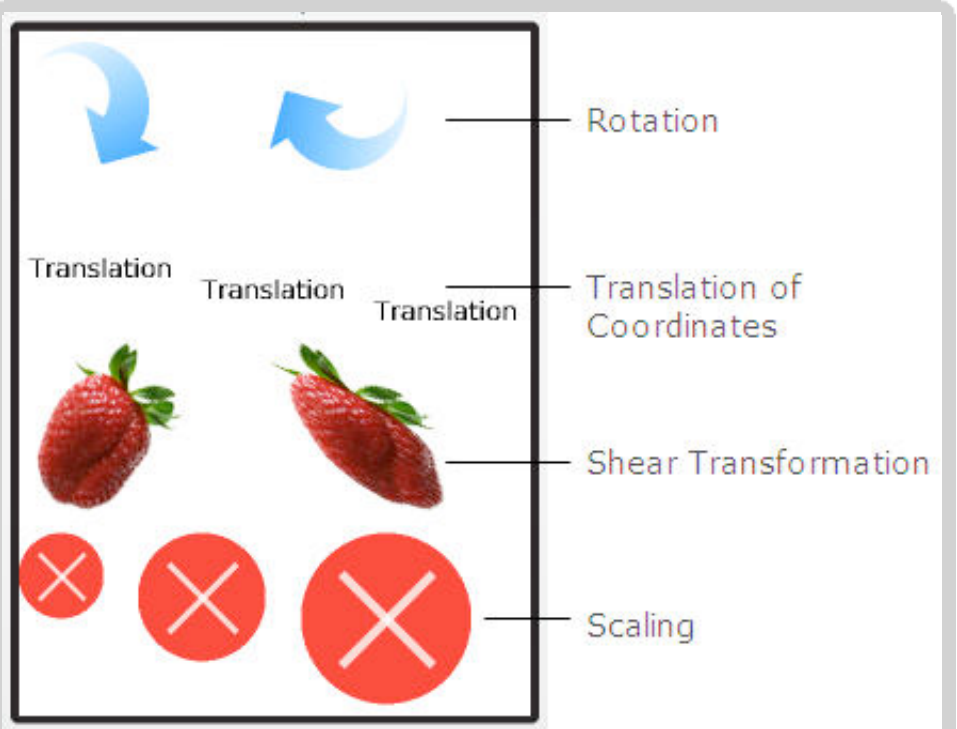

Figure 8: Transformations

Find the complete code of this example in the `transform.fx` file. Transformations are applied to the demo in Presenting UI Objects in a Graphical Scene, and to the Ball demo of this lesson.

Layout

The following screen capture shows methods of laying out UI elements using the `javafx.scene.layout` package.

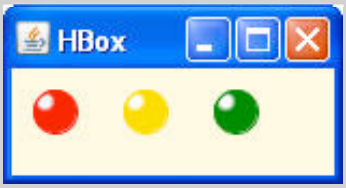

Figure 9: Horizontal Layout

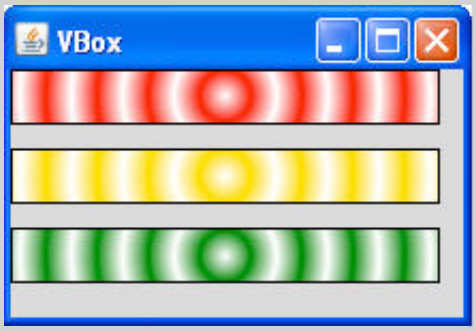

Figure 10: Vertical Layout

Find the complete code of these examples in the `hbox.fx` and `vbox.fx` files. Find the detailed description of the layout mechanism in Laying Out GUI Elements.

Desktop Profile

- User Interface Elements
- Effects
- Cursors

User Interface Elements

The following screen capture shows the standard UI components you can create using the `javafx.ext.swing` package.

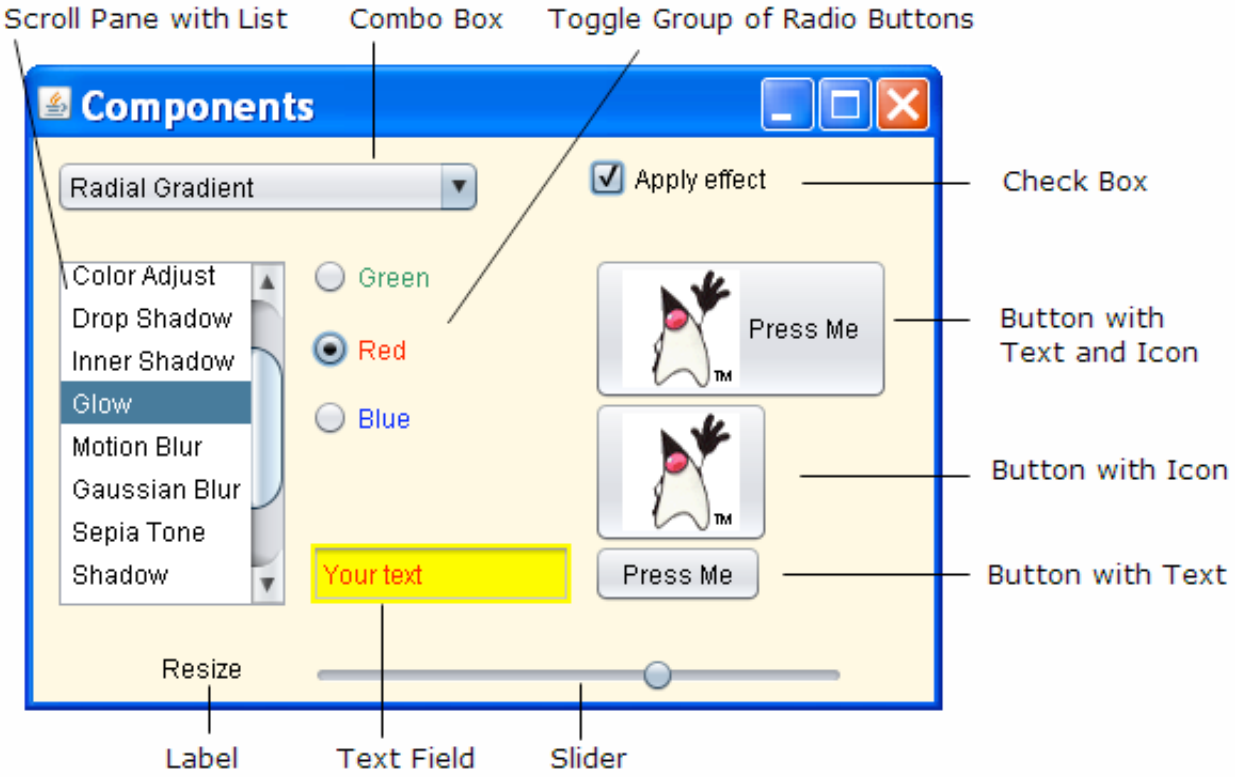

Figure 11: User Interface Components

Find the complete code of this application in the `components.fx` file. UI components are used in the demos of the following lessons: Applying Data Binding to UI Objects and Laying Out GUI Elements.

Effects

The following window is displayed when you run the compiled code in the `effects.fx` file. This window shows effects that can be applied to the JavaFX UI elements. Note that the text captions on the screen capture are not part of the example code.

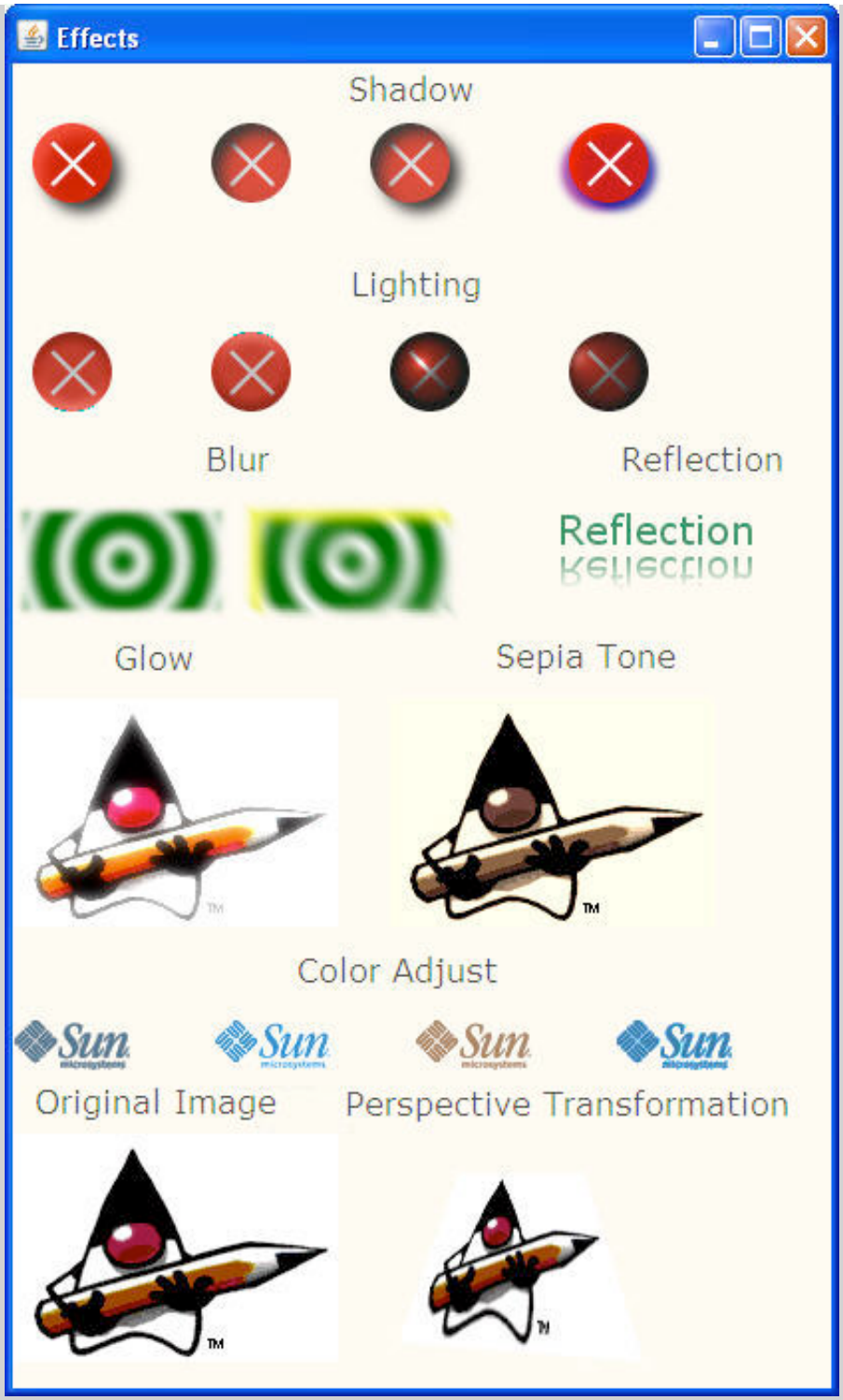

Figure 12: Visual effects

Visual effects are employed in the demos of the following lessons: Creating Graphical Objects, Creating Animated Objects, and Bringing Interactivity to GUI Elements.

Cursors

The following example introduces different views of the cursor you can apply to any UI element in JavaFX. Compile and run the source code in the `cursor.fx` file, then move the mouse cursor from one graphical object to another to explore various cursor views. Note that the text captions on the screen capture are not part of the example code.

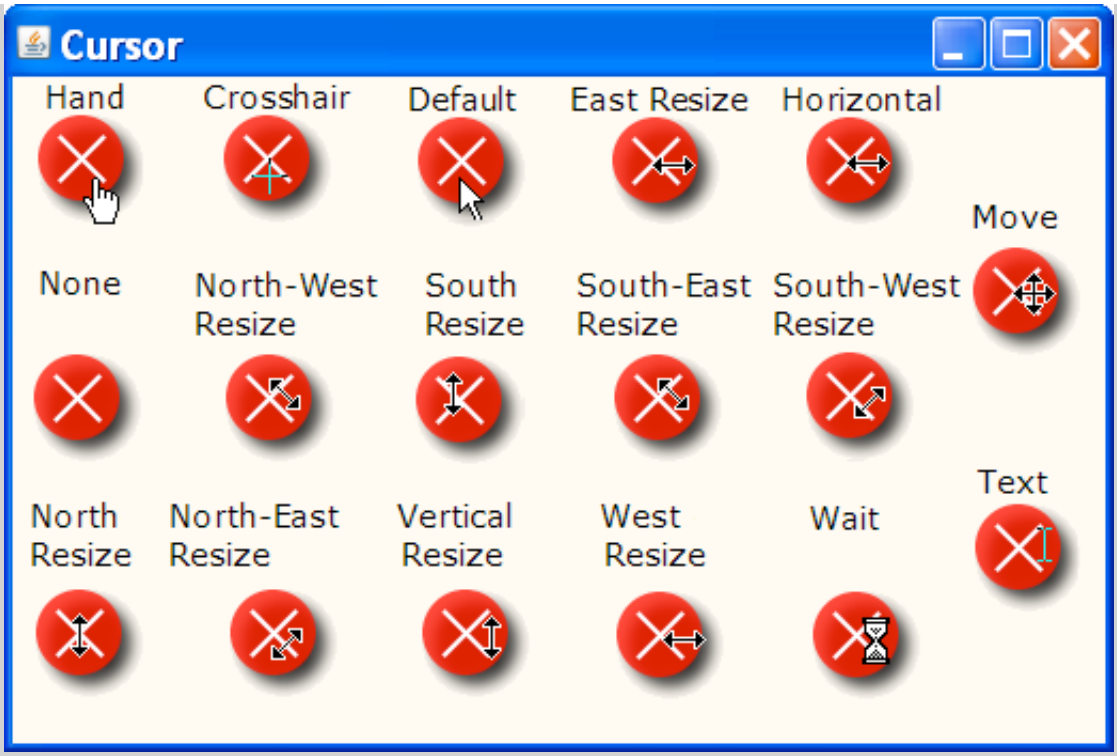

Figure 13: Cursor Styles

Find more information about how to apply the specific cursor style to a graphical object in Bringing Interactivity to GUI Elements.

Conclusion

This lesson briefly introduced the basic GUI features available through the JavaFX SDK API. Refer to the API documentation for more details about the packages and classes used in the examples. Proceed with the next lessons of the tutorial to explore the JavaFX SDK capabilities in depth.

http://java.sun.com/javafx/1/tutorials/ui/syntax/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 2: Using Declarative Syntax

Download tutorial

Are you familiar with declarative programming? JavaFX Script uses this simple but powerful coding style. This lesson shows you how easy it is to use declarative statements through the creation of a simple GUI application. For more information, refer to Writing Scripts, Using Objects, and Writing Your Own Classes in Learning the JavaFX Script Programming Language.

Contents

As you have already read in Learning the JavaFX Script Programming Language, JavaFX Script uses a declarative approach to programming. Declaring is convenient when you create an application's UI because the structure of declared objects in the code reflects the visual structure of the scene graph, and this enables you to understand and maintain the code easily. For more information about the scene graph, see Presenting UI Objects in a Graphical Scene. To help you understand this approach, in this lesson you will follow a step-by-step process to create a sample JavaFX Script application that renders a green rounded rectangle, and a white circle with red outline on the top of the rectangle. Both objects are placed on a window titled "Declaring Is Easy!".

The application is created using the common profile API and can be run both on mobile devices and desktop platforms. If you want to learn more about the desktop platform API, refer to the JavaFX API and to the following chapters of the GUI tutorial. The screen captures provided in this lesson are taken from a desktop application.

The following window is displayed when you run the application.

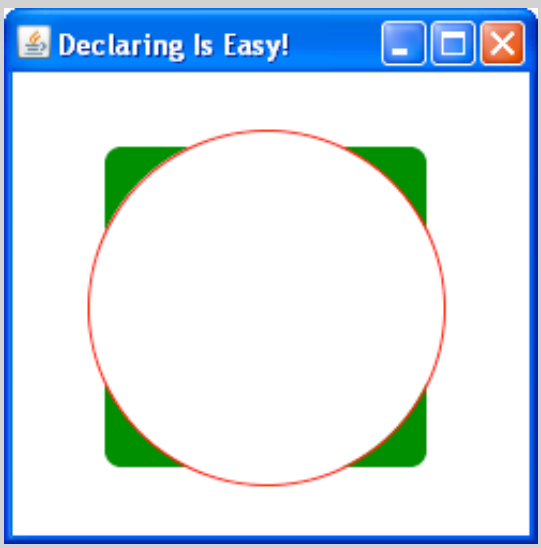

Figure 1: Complete application

By following the steps below, you will learn how to use declarative statements as you build the application.

Create a file with an `.fx` extension, for example `Declaring.fx`. Avoid using file names that match the names of existing classes, instance variables, or reserved words because this leads to errors during compilation. For more information about existing classes, variables, and reserved words, see JavaFX Script API and Learning the JavaFX Script Programming Language.

Adding Necessary Imports

Add imports to the .fx file to make sure the application can access the necessary classes.

```
import javafx.stage.Stage;                      //required to render
                                                //a window
import javafx.scene.Scene;                      //required to display
                                                //a circle and rectangle
                                                //on a window
import javafx.scene.shape.Rectangle;            //required to
                                                //render the rectangle
import javafx.scene.paint.Color;                //required to fill and stroke
                                                //the rectangle and
                                                //circle with color
import javafx.scene.shape.Circle;               //required to render the circle
```

Creating an Application Window

In order to display the graphics, first create a window.

> Note: For the mobile version of the application, this step is required to define the scene.

To create a window:

1. Specify the `Stage` object literal and the `title` variable. Stage is required to render any object.

```
Stage {
    title: "Declaring Is Easy!"
}
```

The word to the left of the colon: `title` is called an *instance variable.* Refer to the `Stage` documentation for a complete list of available variables. The `title` puts the 'Declaring Is Easy' phrase on the top border of the window in case you run the demo on the desktop platform. For more information on object literals, classes, and instance variables in JavaFX Script, see Writing Scripts and Using Objects in Learning the JavaFX Script Programming Language.

Setting the Scene

Within the stage, set the scene to hold `Node` objects such as a circle or a rectangle. Create a `Scene` using the following code:

```
Stage {
    ...
    scene: Scene {
        width: 249
        height: 251
        content: [ ]
    }
}
```

The scene is a root area where you place objects of the node type. There are many different kinds of nodes, such as graphical objects, text, and GUI components. For more information about nodes and `Scene` class, see the Presenting UI Objects in a Graphical Scene lesson and JavaFX Script API.

The scene has a `content` variable that is used to hold the nodes.Its `width` and `height` variables are used to specify the dimension of the scene in pixels. This step is required only for the desktop version of the demo to specify the dimension of the application window.

When you run the code you have defined so far, you see the following window.

Figure 2: Window with the scene

Note: The content of the window becomes filled with white because white is the default fill color for a scene. The scene is placed on top of the window.

Creating a Rectangle

To declare a rectangle within the `content`, use this code:

```
content: [
    Rectangle {
        x: 45
        y: 35
        width: 150
        height: 150
        arcWidth: 15
        arcHeight: 15
        fill: Color.GREEN
    }
]
```

Note: In JavaFX Script, by convention you specify one instance variable per line as shown in the preceding example. However, to optimize the code you can specify all the variables in a single line without changing the essence of code, for example:

```
content: [
    Rectangle {x: 45 y: 35 width: 150 height: 150 arcWidth: 15 }
]
```

You can also use commas to separate the instance variables and make the code more readable:

```
content: [
    Rectangle {x: 45, y: 35, width: 150, height: 150, arcWidth: 15 }
]
```

The `x` and `y` instance variables specify the pixel location of the rectangle, `arcWidth` and `arcHeight` define the roundness of corners, and the `fill` variable defines the color that fills the rectangle. You saw the size variables `width` and `height` when you defined the dimension of the scene.

Note: In the preceding code sample, you explicitly declare the green fill color, however you can declare a web code that represents this color. To specify the green fill color using its code, declare:

```
fill: Color.web("#008000")
```

As a result, this code creates a rectangle positioned with the left-top corner at 45,35. The rectangle has the size of 150 by 150 pixels, a corner roundness of 15, and is filled with green. For more information about the `Rectangle` class, see the JavaFX Script API . The following graphic illustrates the application window in this step.

Figure 3: Green rounded rectangle

Creating a Circle

Declare a circle on the top of the green rectangle and set its style using the following code:

```
content: [
```

```
    Rectangle{
        ...
    },
    Circle{
        centerX: 118
        centerY: 110
        radius: 83
        fill: Color.WHITE
        stroke: Color.RED
    }
]
```

Because the rectangle is declared before any other objects, it is painted first. The rectangle will be behind any other objects painted later.

This code uses a `Circle` object literal to create an instance of the `Circle` class. Circle has five instance variables that define its state, including the X and Y position on a window, radius, fill, and stroke colors. As a result, this code creates a circle with a radius of 83, positioned with its center at 118,110, filled with white and stroked with red. For more information about the `Circle` class, see the JavaFX Script API.

Running the Example

Now you are ready to run the whole example. The following code is a complete `Declaring.fx` file:

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;

Stage {
    title: "Declaring Is Easy!"
    scene: Scene {
        width: 249
        height: 251
        content: [
            Rectangle {
                x: 45 y: 35
                width: 150 height: 150
                arcWidth: 15 arcHeight: 15
                fill: Color.GREEN
            },
            Circle {
                centerX: 118
                centerY: 110
                radius: 83
                fill: Color.WHITE
                stroke: Color.RED
            }
        ]//Content
    }//Scene
}//Stage
```

The following window appears when you run the code.

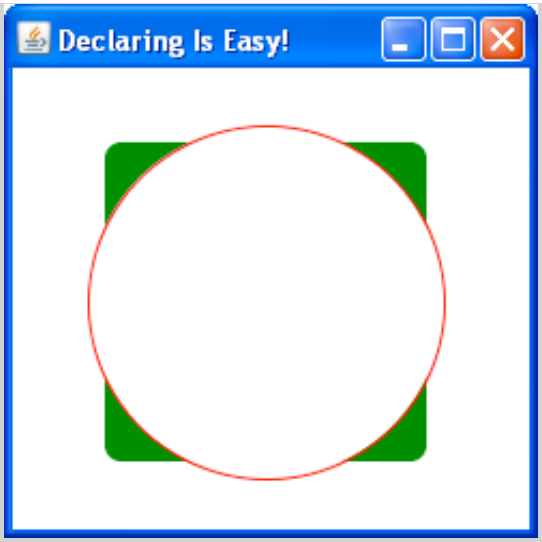

Figure 4: Complete application

Changing the Code and Running the Example

Place the circle underneath the square. To do so, switch the order of the circle and square using the following code:

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;

Stage {
    title: "Declaring Is Easy!"
    scene: Scene {
        width: 249
        height: 251
        content: [
            Circle {
                centerX: 118
                centerY: 110
                radius: 83
                fill: Color.WHITE
                stroke: Color.RED
            },
            Rectangle {
                x: 45 y: 35
                width: 150 height: 150
                arcWidth: 15 arcHeight: 15
                fill: Color.GREEN
            }
        ]//Content
    }//Scene
}//Stage
```

Compile and run the application. The rectangle is now on top of the circle.

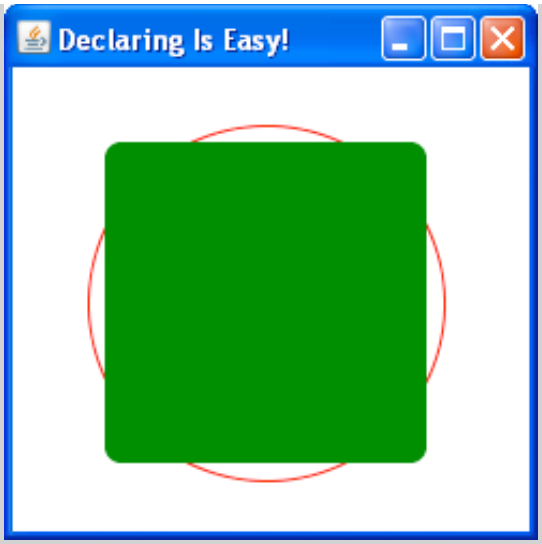

Figure 5: Switched order of objects

Note: You can use layout approaches supported by JavaFX Script to simplify the layout of objects. For more information about layout approaches, see Laying Out GUI Elements.

Conclusion

As this lesson shows, the declarative syntax simplifies the creation of graphics and makes the code easy to read and maintain. The order of elements you declare in the code matches the order in which they appear in the application.

« Previous 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | Next »

http://java.sun.com/javafx/1/tutorials/ui/nodes/ Feb 10, 2009

Building GUI Applications With JavaFX

Lesson 3: Presenting UI Objects in a Graphical Scene

Download tutorial

« Previous 1 | 2 | **3** | 4 | 5 | 6 | 7 | 8 | Next »

This lesson explains the node architecture and scene graph that underly the JavaFX Script Programming Language, and includes information on the `Scene`, `Node`, and `Group` classes. In this lesson you will build a graphical scene, create a group of nodes, and apply a transformation to the group. Refer to Using Declarative Syntax for more information on the concept of declarative syntax.

Contents

The JavaFX Script Programming language is based on a scene graph. The scene graph is a tree-like data structure which defines a hierarchy of graphical objects in a scene. A single element in the scene graph is called a node. Each node has one parent except for the root node, which has no parent. Each node is either a leaf node or a branch. A leaf node has no children. A branch node has zero or more children.

JavaFX nodes handle different types of content such as UI components, shapes, text, images, and media. Nodes can be transformed and animated. You can also apply various effects to nodes.

In this lesson, you will create an application with three nodes: a circle, text, and an image, as shown below.

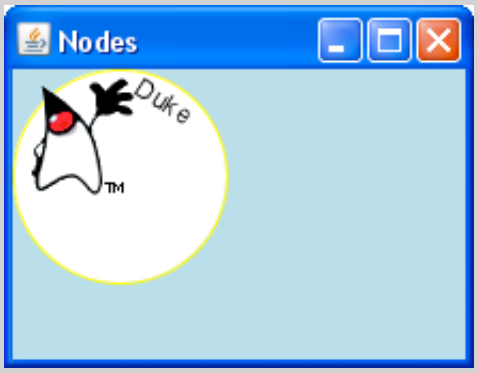

Figure 1: The Nodes example

The application is created using the common profile API and can be run both on mobile devices and desktop platforms. If you want to learn more about the desktop platform API, refer to the JavaFX API. The screen captures provided in this lesson are taken from a desktop application.

JavaFX renders everything on a scene. You can think of the scene as a drawing surface for graphical content. The scene is a container that holds the scene graph nodes.

In any JavaFX GUI application, you create a scene and add nodes to it. You can modify the graphical scene by applying effects,

transformations, and animation. The JavaFX runtime takes care of any changes in the graphical scene and performs any necessary repaints for you.

The `javafx.scene.Node` class is the base class for the scene graph nodes. All other node classes, for example `javafx.scene.shape.Circle`, inherit from the `Node` class. For a complete list of instance variable and functions, see the API documentation for the `Node` class.

The `Node` class defines a local coordinate system in which the X coordinate increases to the right, and the Y coordinate increases downwards.

Nodes can be changed by applying transformations such as translation, rotation, scaling, and shearing. For example, a translation moves the origin of the node's coordinate system along either the X or Y axis, or both. To define the translation, set the values for the `translateX` or `translateY` variables or both.

JavaFX provides powerful support for effects available through the `javafx.scene.effect` and `javafx.scene.effect.light` packages. You can see some of the applied effects as well as transformations in Quick JavaFX GUI Overview. Note that the packages are available only in the desktop profile API.

Nodes can receive mouse and keyboard events. You can define functions to be notified when such events occur. For details, see Bringing Interactivity to GUI Elements.

Nodes can be grouped together and treated as a single entity. If you need to provide common behavior for several nodes, group them, and define the required behavior for the whole group. The `javafx.scene.Group` class represents a group of nodes.

Now you will create a simple application as shown in Figure 1. The graphical scene of this application contains three nodes displayed below on separate windows. They are a shape object (a circle), text, and an image.

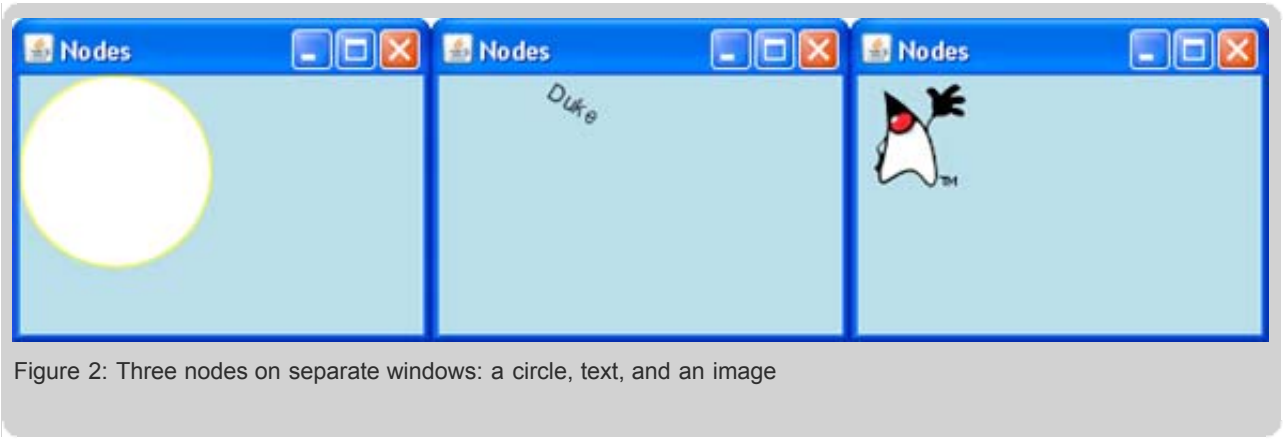

Figure 2: Three nodes on separate windows: a circle, text, and an image

First you will add the nodes to the scene as separate nodes. Then you will group them and apply a transformation to the whole group.

Creating an Application Window

Create an application window with the title "Nodes". For details, see Using Declarative Syntax. The following code creates the window.

> Note: For the mobile version of the application, this step is required to define the scene.

```
import javafx.stage.Stage;

Stage {
    title: "Nodes"
}
```

Creating a Scene

A scene is declared using the Scene object literal. You will set the scene's width to 220 pixels, its height to 170 pixels, and give it a light blue background.

1. Add import statements for the `javafx.scene.Scene` and `javafx.scene.paint.Color` classes.
2. Declare the `Scene` object literal.
3. Define the `fill` variable to set the background for the scene.
4. Define the `width`, and `height` attributes of the `Scene` object. This step is required only for the desktop version of the demo to specify the dimension of the application window. For details, see the Using Declarative Syntax lesson.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;

Stage {
    title: "Nodes"
    scene: Scene {
        fill: Color.LIGHTBLUE
        width: 220
        height: 170
    }
}
```

This code produces the following output.

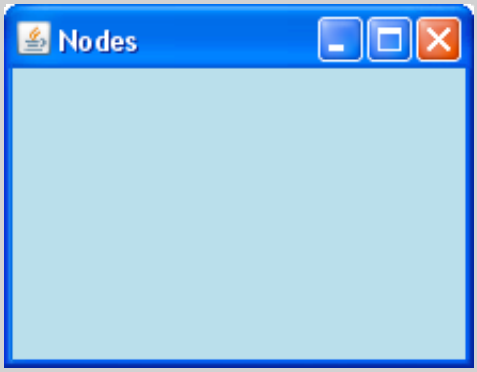

Figure 3: A scene with light blue background

Adding the First Node

You add a node to the scene by declaring this node as an element of the `content` of the `scene`. The `content` variable of the `scene`, which is a sequence of nodes, defines the graphical content of your application.

The first node is a circle. For details on the `Circle` class, see Using Declarative Syntax. You will paint the boundary of the circle with yellow color.

1. Import the `javafx.scene.shape.Circle` class.
2. Define the `content` variable of the `scene`.
3. Add the `Circle` object literal to the `content` variable.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
```

```
import javafx.scene.shape.Circle;

Stage {
    title: "Nodes"
    scene: Scene {
        fill: Color.LIGHTBLUE
        width: 220
        height: 170
        content: Circle {
            centerX: 50   centerY: 50   radius: 50
            stroke: Color.YELLOW
            fill: Color.WHITE
        }
    }
}
```

The modified code gives you the following output.

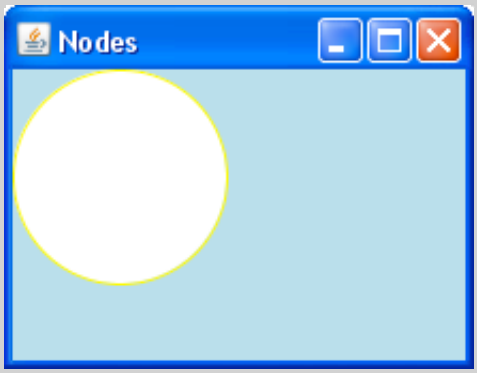

Figure 4: A scene with a circle node

Adding a Text Node

1. Add an import statement for the `javafx.text.Text` class.
2. Add the `Text` object literal to the `content` variable.

By default the text node will be placed at the point (0,0), which means that the left bottom point of the first character will be placed at (0,0). For this reason, the text is not visible in the application window when the code is compiled and run. In the next step, the default location will be changed so that text is visible.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Text;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: [
            Circle {
```

```
                centerX: 50   centerY: 50   radius: 50
                stroke: Color.YELLOW
                fill: Color.WHITE
            },
            Text {
                content: "Duke"
            }
        ]//Content
    }//Scene
}//Stage
```

Use square brackets to specify a sequence of nodes and commas to separate its elements.

Applying a Transformation

You can change the default location by applying a rotation transformation. The rotation is specified by an anchor point and an angle. The node will be rotated clockwise around the anchor point by the specified angle.

To calculate the necessary values, look at Figure 5. If you take the point (10, 100) as an anchor point and draw a circle with a radius equal to the distance to the left bottom point of the text node, then part of this circle falls inside the circle node. Moving the text node along this circle by 33 degrees clockwise gives the result shown in Figure 5.

1. Add an import statement for the `javafx.scene.transform.Transform` class.
2. Define the `transforms` variable of the text node to rotate the node by 33 degrees around the point (10,100).

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Text;
import javafx.scene.transform.Transform;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: [
            Circle {
                centerX: 50   centerY: 50   radius: 50
                stroke: Color.YELLOW
                fill: Color.WHITE
            },
            Text {
                transforms: Transform.rotate(33, 10, 100)
                    content: "Duke"
            }
        ]//Content
    }//Scene
}//Stage
```

The modified code provides the following output.

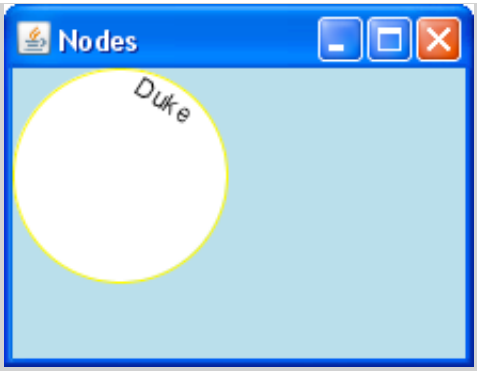

Figure 5: A scene with a circle and text nodes

Adding the Image Node

JavaFX applications can display images that are stored either on the Internet or in the local directory of your computer. The images are displayed using the `ImageView` and `Image` classes, and the `url` variable which points to the location of the image. In case you use an image from the Internet, its `url` variable which indicates the URL, is specified as a URI. In case you refer to an image from the local directory, its `url` variable which indicates the path to the directory, is specified using the `__DIR__` variable.

If you keep an image on the Internet, you need an Internet connection in order to display it in the application.

The example from this section uses an image of Duke from the Java Tutorials stored in the local directory. The example specifies the image using the `__DIR__` variable. By default, it points to the current directory, so make sure that the image is located in the same directory as the application's compiled classes. To run the application on the mobile emulator, make sure that the image is packed into the application jar file along with the compiled classes.

By default, the left upper point of the image node is placed in the point (0,0). The dimensions of this image fit exactly into the area over the circle node.

For more information about the ImageView and Image classes, see the JavaFX API. For more details on the use of images, see Creating Animated Objects.

1. Add import statements for the `Image` and `ImageView` classes from the `javafx.scene.image` package.
2. A dd the `ImageView` object literal to the `content` variable.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Text;
import javafx.scene.transform.Transform;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: [
            Circle {
                centerX: 50   centerY: 50   radius: 50
                stroke: Color.YELLOW
                fill: Color.WHITE
```

```
            },
            Text {
                transforms: Transform.rotate(33, 10, 100)
                    content: "Duke"
            },
                ImageView {
                image: Image {url: "{__DIR__}dukewave.png"}
            }
        ]//Content
    }//Scene
}//Stage
```

You created an application whose graphical scene contains three nodes. The output is shown in the following image.

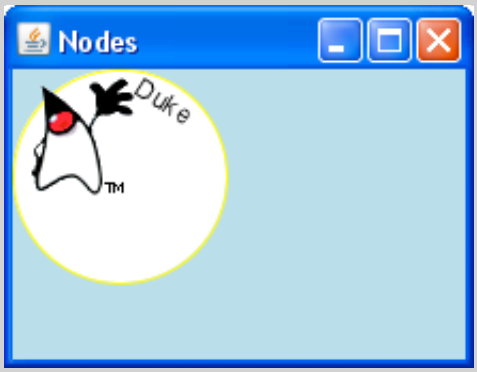

Figure 6: A scene with three nodes

Grouping the Nodes

Now add the nodes to a group and then add the group to the `content` variable of the `scene`.

1. Add an import statement for the `javafx.scene.Group` class.
2. Modify the declaration of the `content` variable for the `scene` so that it contains the `Group` object literal.
3. Move all nodes to the `content` variable of the `Group`. The code appears as follows.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Text;
import javafx.scene.transform.Transform;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.Group;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: Group {
            content: [
                Circle {
                    centerX: 50   centerY: 50   radius: 50
```

```
                    stroke: Color.YELLOW
                    fill: Color.WHITE
                },
                Text {
                    transforms: Transform.rotate(33, 10, 100)
                    content: "Duke"
                },
                ImageView {
                    image: Image {url: "{__DIR__}dukewave.png"}
                }//ImageView
            ]//Content
        }//Group
    }//Scene
}//Stage
```

Note the importance of the order in which you add objects to your group. This order defines how those objects are laid out. If you add the circle node last (after the text and image nodes), then the circle will be drawn over the two other objects. Because the circle has a fill color, these nodes will not be seen.

Applying a Transformation to the Group

Finally, define the translation for the group of nodes to move the group to the center of the window as shown in the following code.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Text;
import javafx.scene.transform.Transform;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.Group;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: Group {
            translateX: 55
            translateY: 10
            content: [
                Circle {
                    centerX: 50   centerY: 50   radius: 50
                    stroke: Color.YELLOW
                    fill: Color.WHITE
                },
                Text {
                    transforms: Transform.rotate(33, 10, 100)
                    content: "Duke"
                },
                ImageView {
                    image: Image {url: "{__DIR__}dukewave.png"}
                }//ImageView
            ]//Content
        }//Group
    }//Scene
```

```
}//Stage
```

This modification shifts all three nodes simultaneously as displayed in the following image.

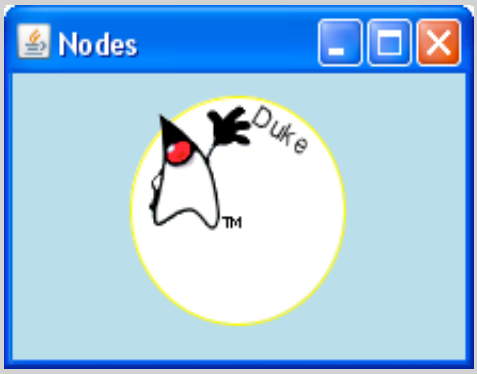

Figure 7: A group of nodes shifted to the center of the window

For your reference, here is the complete code of this example application.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.text.Text;
import javafx.scene.transform.Transform;

Stage {
    title: "Nodes"
    scene: Scene {
        width: 220
        height: 170
        fill: Color.LIGHTBLUE
        content: Group {
            translateX: 55
            translateY: 10
            content: [
                Circle {
                    centerX: 50   centerY: 50   radius: 50
                    stroke: Color.YELLOW
                    fill: Color.WHITE
                },
                Text {
                    transforms: Transform.rotate(33, 10, 100)
                    content: "Duke"
                },
                ImageView {
                    image: Image {url: "{__DIR__}dukewave.png"}
                }//ImageView
           ]//Content
        }//Group
    }//Scene
```

```
}//Stage
```

Conclusion

In this lesson you learned how to build a graphical scene, add nodes to the scene, group nodes, and apply a transformation to the group. Now you can proceed with creating more sophisticated graphical applications.

« Previous 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | Next »

http://java.sun.com/javafx/1/tutorials/ui/graphics/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 4: Creating Graphical Objects

Download tutorial

The Quick JavaFX GUI Overview introduced the rich set of built-in graphics, components, and effects available in JavaFX. This lesson will show you how to use these elements to customize or create even more rich visuals for your application.

In this lesson, you combine a few simple elements to create a button. Once you understand the concept of combining the various elements, you can use it to create more elements.

The lesson is divided into two sections: Common Profile and Desktop Profile. The Common Profile section describes how to create graphics for the application that draws a button and runs both on mobile devices and desktop platforms. The Desktop Profile section describes how to add a desktop specific reflection and background gradient effect to this graphic to make it look even more appealing.

Common Profile

You have already read enough about the JavaFX language to start creating more sophisticated graphical objects. This section describes the typical process of creating graphics using JavaFX Script. In this lesson you will create a design for an audio player including a record button as shown in the following picture.

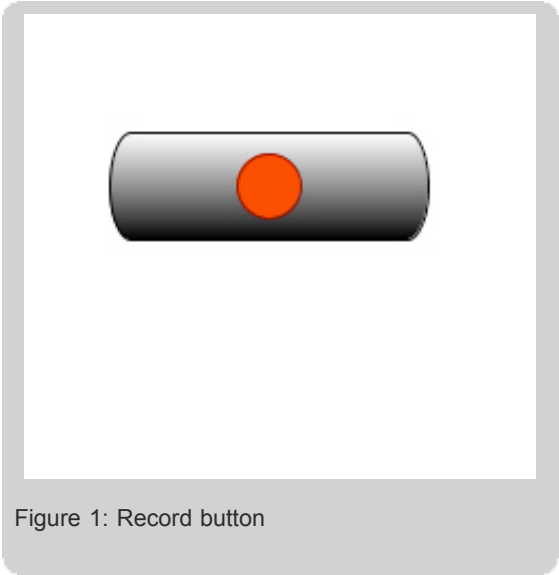

Figure 1: Record button

To create the button, you choose from a variety of JavaFX objects and features described in the The Quick JavaFX GUI Overview lesson. Those objects and features are: a rectangle, a circle, and a gradient effect. Then you combine those features to achieve the desired graphics for the button.

> Note: This lesson uses a limited number of graphical features from the JavaFX language. You can combine other effects and features to create UI elements such as slider bars, progress bars, buttons, and search fields.

Create a file with an `.fx` extension, for example `FXRecordButton.fx`. Avoid using file names that match the names of existing classes, instance variables, or reserved words because this leads to errors during compilation. For more information about existing classes, variables, and reserved words, see JavaFX Script API and Learning the JavaFX Script Programming Language.

Creating an Application Window

In order to display the graphics, first create a window.

> Note: For the mobile version of the application, this step is required to define the scene.

To create a window:

1. Add an import statement for the `javafx.stage.Stage` class.
2. Declare the `Stage` object literal.
3. Define the `title` attribute of the `Stage` object. For details, see the Using Declarative Syntax lesson.

Here is the code:

```
import javafx.stage.Stage;                          //required to
                                                    //render a window

    Stage {
        title: "JavaFX Record Button"
    }
```

Setting the Scene

Within the `Stage`, set the scene to hold `Node` objects such as a circle or a rectangle, and fill it with the linear gradient.

To define the scene and fill it with white:

1. Add import statements for the `javafx.scene.Scene` and for the `javafx.scene.paint.Color` classes.
2. Declare the `Scene` object literal.
3. Define the `fill` variable of the `Scene` object. The fill variable is used to fill the background with either a color or a gradient..
4. Specify `content` variable within the `Scene`. The `content` variable is used to hold objects of the `Scene`.
5. Define the `width`, and `height` attributes of the `Scene` object. This step is required only for the desktop version of the demo to specify the dimension of the application window. For details, see the Using Declarative Syntax lesson.

The following code sets the scene and fills it with white:

```
import javafx.scene.Scene;                          //required to
                                                    //display objects of Node
                                                    //type such as a circle
                                                    //and rectangle
javafx.scene.paint.Color                            //required to draw a gradient


Stage {
    ...
    scene: Scene {
        width: 241
        height: 217
        fill: Color.WHITE
```

```
        content: [
                //objects that appear on the scene
            ]//Content
    }//Scene
}//Stage
```

For more information about nodes and `Scene` class, see the Presenting UI Objects in a Graphical Scene lesson and JavaFX Script API.

Specifying Objects

Now you can proceed to specify the objects within the scene. The button consists of a rectangle and a circle. Specify the rectangle and the circle within a group.

To specify objects:

1. Add an import statement for the `javafx.scene.Group` class.
2. Declare the `Group` object literal.
3. Specify `content` variable within the `Group`. The `content` variable is used to hold objects of the `Group`.

The code looks like this:

```
import javafx.scene.Group;                          //required to group
                                                    //objects to be able
                                                    //to operate with them as a unit

Stage {
    ...
    scene: Scene {
        ...
        content: [
            Group {
                content: []
            }//Group
        ]//Content
    }//Scene
}//Stage
```

For more information about the `Group` class, see JavaFX Script API.

Creating a Rectangle

To create a button outline, declare a rectangle as part of the `Group` content.

To create a rectangle:

1. Add the `javafx.scene.shape.Rectangle` import.
2. Declare the `Rectangle` object literal and its variables. For more information about the `Rectangle` class, see the Using Declarative Syntax lesson and the JavaFX Script API.

Here is the code that accomplishes these two steps:

```
import javafx.scene.shape.Rectangle;            //required to render a rectangle

    content: [
        Group {
```

```
            content: [
                Rectangle {
                    x: 40 y: 55 width: 150 height: 50
                    arcWidth: 20 arcHeight: 55 stroke: Color.BLACK
                    fill: null
                }//Rectangle
            ]//Content
            }//Group
    ]//Content
```

The preceding code results in the following screen capture.

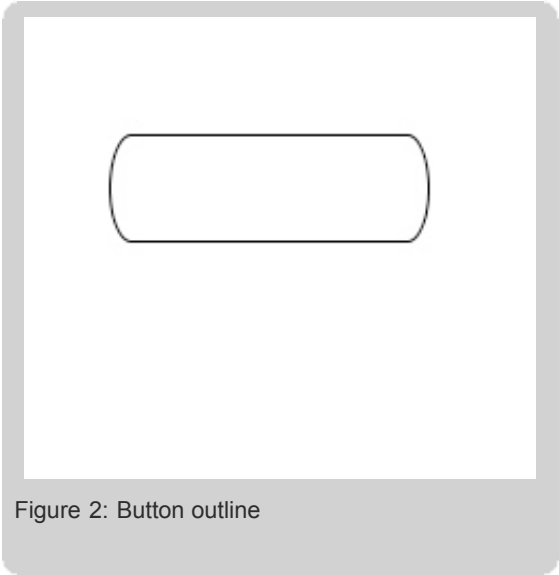

Figure 2: Button outline

Filling the Rectangle

Fill the rectangle with a shaded fill from black to white using the `fill` variable. This type of even shading between two colors is called a *linear gradient*. The effect is produced using the `javafx.scene.paint.LinearGradient` class.

Here is the code to produce that effect:

```
import javafx.scene.paint.LinearGradient;              //required to render a gradien

Rectangle {
    ...
    fill: LinearGradient {
        startX: 0.0, startY: 0.0, endX: 0.0, endY: 1.0, proportional: true
        stops: [
            Stop {offset: 0.0 color: Color.WHITE},
            Stop {offset: 1.0 color: Color.BLACK}
        ]//Stops
    }//Fill
}//Rectangle
```

As a result, this code creates a rectangle with the left-top corner at 40,55. The rectangle is placed in the center of the window and has the size of 150 by 50 pixels, a corner roundness of 20 and 55, and is filled with a black and white even linear gradient. For more information about the linear gradient, see JavaFX Script API.

The screen capture of the application at this step is shown in the following picture.

Figure 3: Button filled with gradient

Adding the "Record" Indicator

Now you can add the red "Record" indicator to the button.

To add the red indicator to the button:

1. Add the `javafx.scene.shape.Circle` import.
2. Declare the circle and its variables using the `Circle` class. For more information about the `Circle` class, see the Using Declarative Syntax lesson and the JavaFX Script API.

> Note: The circle is also part of the `Group` content definition.

Here's the code that adds the red indicator:

```
import javafx.scene.shape.Circle;              //required to render a circle

content: [
    Group {
        content: [
            Rectangle {
                ...
            },
            Circle {
                centerX: 115 centerY: 80 radius: 15
                fill: Color.web("#ff3300") stroke: Color.DARKRED
            }//Circle
        ]//Content
    }//Group
]//Content
```

As a result, this code creates a circle with the center at 115,80 and radius 15. The circle is placed in the center of the rectangle. The circle is filled with red and stroked with dark red. The following screen capture shows a rectangle with a red circle on top.

Figure 4: Button with a "Record" indicator

Running the Example

Now you are ready to run the whole example. The following code is a complete `FXRecordButton.fx` file:

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.paint.Color;
import javafx.scene.Group;
import javafx.scene.shape.Rectangle;
import javafx.scene.shape.Circle;
import javafx.scene.effect.Reflection;

Stage {
    title: "JavaFX Record Button"
    scene: Scene {
        width: 249
        height: 251
        fill: Color.WHITE
        content: [
            Group{
                content: [
                    Rectangle {
                        x: 40 y: 55 width: 150 height: 50
                        arcWidth: 20 arcHeight: 55 stroke: Color.BLACK
                        fill: LinearGradient {
                        startX: 0.0, startY: 0.0, endX: 0.0, endY: 1.0, proportional
                        stops: [
                            Stop {offset: 0.0 color: Color.WHITE},
                            Stop {offset: 1.0 color: Color.BLACK}
                        ]
                    }
                },
                    Circle {
                         centerX: 115 centerY: 80 radius: 15
                         fill: Color.web("#ff3300") stroke: Color.DARKRED
                    }//Circle
                ]//Content
            }//Group
        ]//Content
    }//Scene
```

```
}//Stage
```

The result of the running application is shown on the following picture.

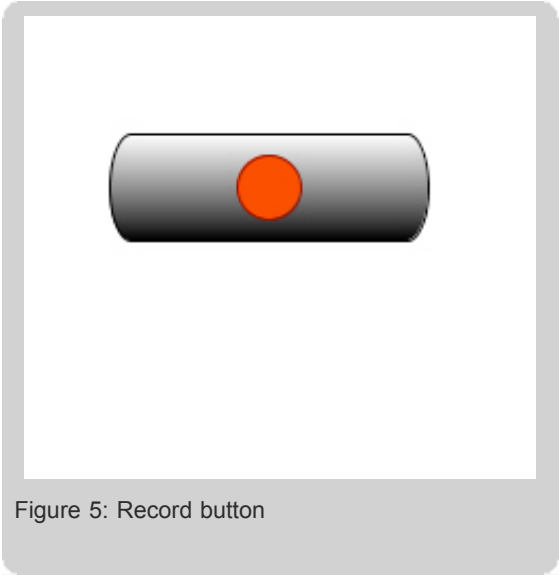

Figure 5: Record button

Desktop Profile

In order to visually enhance the application and to use the power of the desktop profile, you can add a gradient to the scene and a reflection effect to the button.

> Note: The application from this section is based on a demo you created in the Common Profile section.

Adding a Gradient to the Scene

In this step you fill the scene with the gradient.

To apply the gradient

1. Add import statements for the `javafx.scene.paint.LinearGradient`, `javafx.scene.paint.Stop`, `javafx.scene.paint.Color` classes.
2. Re -define the `fill` variable of the `Scene` object.

The following code fills the scene with an even black and white linear gradient:

```
import javafx.scene.paint.LinearGradient;         //required to fill
                                                  //objects with a linear gradient
import javafx.scene.paint.Stop;                   //required to specify
                                                  //colors and offset of the
                                                  //linear gradient
import javafx.scene.paint.Color;                  //required to fill
                                                  //and stroke objects with color

Stage {
    ...
    scene: Scene {
        fill: LinearGradient {
            startX: 0.0, startY: 0.0, endX: 0.0, endY: 1.0, proportional: true
            stops: [
                Stop {offset: 0.0 color: Color.WHITE},
```

```
                Stop {offset: 1.0 color: Color.BLACK}
            ]
        }
        content: [
                //objects that appear on the scene
            ]//Content
    }//Scene
}//Stage
```

- To fill the scene with the gradient, use the `fill` instance variable and specify the `LinearGradient` object literal as its value. The `LinearGradient` has instance variables that define the direction, the size, the colors and the style of the gradient.
- `startX`, `startY`, `endX`, and `endY` instance variables control the horizontal and vertical direction and the size of the gradient. Each pair - `startX`, `startY` and `endX`, `endY` define the coordinates of start and end points of the gradient. If an end value of a pair is smaller than the start value of the same pair, then the direction of the gradient is reversed.

  Note: The values of this variable depend on the `proportional` variable described in the following paragraph.

- The `proportional` variable defines whether the values of `startX`, `startY`, `endX`, and `endY` are scaled or not. If the `proportional` variable is set to `true` then the start and end points of the gradient should be specified relative to the window square (0.0 — 1.0), and will be stretched across the window. If the `proportional` variable is set to `false`, then the start and end points should be specified as absolute pixel values and the gradient will not be stretched at all.

  For example, if the `startY` is set to 30, `endY` set to 100, both `startX` and `startY` are set to 0, and the `proportional` is set to `false`, then the Y start point of the gradient will be a point 15 pixels below the title bar of the window and the Y end point of the gradient will be a point 100 pixels below. If the `startY` is set to 0.5, `endY` set to 1.0, the `proportional` is set to `true`, and `startX` and `endX` are both set to 0, then the Y start point of the gradient will be a point that has a Y value that is 50% of a height of a scene, and the Y end point will be a point that has a value that is 100% of a height of a scene.
- The `stops` is a sequence of `Stop` variables that define how to distribute colors along the gradient. The `offset` variable of `Stop` defines the point where the gradient should be a particular color. The `offset` is specified relative to the scene square and its values should range from 0.0 to 1.0. The `Color` variable defines the color of the gradient. As a value of `Color` you can specify either an explicit color, for example `Color.WHITE`, or a web code for this color, for example, `Color.web("FFFFFF")`. For more information about the linear gradient, see JavaFX Script API.

Applying the Reflection Graphical Effect

In this step you add a reflection of the record button to the background.

To apply the reflection effect:

1. A dd the `javafx.scene.effect.Reflection` import statement.
2. Specify the effect using the `effect` and `Reflection` variables.

While this might *seem* like a difficult step to program, it only requires the following lines of code:

```
import javafx.scene.effect.Reflection;           //required to
                                                 //apply a reflection effect


content: [
    Group {
        content: [
            Rectangle{
                ...
            },
```

```
            Circle {
                ...
            }
        ]
        effect:  Reflection {fraction: 0.9  topOpacity: 0.5 topOffset: 2.5}
    }//Group
]//Stage
```

The `reflection` object literal has a `fraction` instance variable that defines the percent of the button area that is visible in the reflection. The `topOpacity` variable defines the opacity of the reflection, and `topOffset` defines the distance between the bottom of the button and the top of the reflection.

Note: The `fraction` and the `topOpacity` variables can only take values ranging from 0.0 to 1.0.

After you apply the effect and the gradient, the application will look like this:

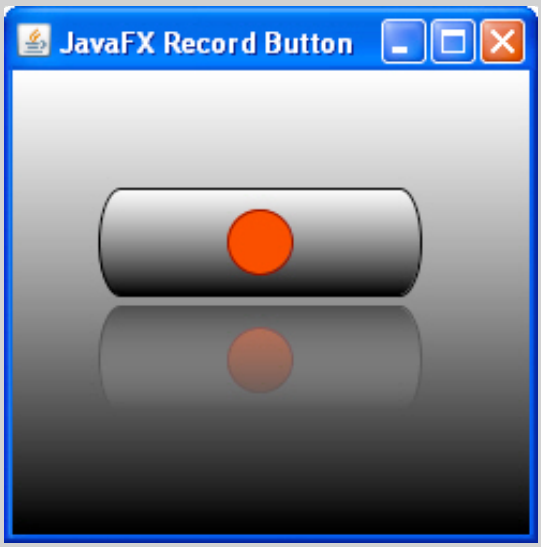

Figure 6: Record button with reflection

For more information on the reflection effect, see JavaFX Script API. For a complete list of graphical effects in JavaFX API, see The Quick JavaFX GUI Overview lesson.

You can find the complete source code for the application in the `FXRecordButton.fx` file.

Conclusion

This lesson shows you how easily a combination of built-in JavaFX SDK effects and objects can be used to create rich visual graphics.

http://java.sun.com/javafx/1/tutorials/ui/binding/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 5: Applying Data Binding to UI Objects

Download tutorial

« Previous 1 | 2 | 3 | 4 | **5** | 6 | 7 | 8 | Next »

This lesson introduces a feature of JavaFX called data binding. With this mechanism, when one variable is changed, another variable is updated according to the relationship that you defined between the two variables. Refer to Data Binding and Triggers, a lesson in Learning the JavaFX Script Programming Language, for more information on the concept of data binding.

In programming you often need to update a certain parameter whenever another parameter changes. In the JavaFX Script Programming language you can accomplish this with the data binding mechanism. You define a relationship between any two variables so that whenever one variable changes the other one is updated. JavaFX keeps track of any changes and makes any necessary updates for you. This relationship, along with the update mechanism, is called data binding. To learn how data binding works, you will create simple applications using the Desktop and Common profiles of the JavaFX API.

Common profile

The binding mechanism can be applied to the common profile, which means it will work fine both in mobile and desktop applications. Consider a palette of all color patterns available through the `javafx.scene.paint.Color` class, an example, that was introduced in Quick JavaFX GUI Overview. Clicking a color on the palette changes the background to the corresponding color.

Figure 1: The color palette run on the mobile emulator

Defining Colors and Constants

In order to create a palette of all color constants available through the JavaFX SDK API, you need to allocate 140 square boxes filled with the corresponding colors.

Perform the following steps to define colors and constants.

1. Define the size of the box and the space between two contiguous boxes.

```
def size = 19;
def space = 3;
```

2. Define the number of columns and rows in the palette.

```
def num_columns  = 10;
def num_rows = 14;
```

3. Define the color names in the sequence `colors`. Use the complete list of color names that correspond to the `Color` class constants.

```
def colors = ["aliceblue", "antiquewhite", ... "yellowgreen"];
```

4. Create a variable to store the color selected from the palette and set it to the "white"

```
var color = "white";
```

Creating an Application Scene

Perform the following steps to create an application window.

1. Add the import statement for the `Stage`, `Scene`, and `Color` classes.
2. Specify the color of the scene using the `fill` instance variable.
3. Define variables of the `Scene` object such as `height`, `width`, and the `title` variable of the `Stage` object, so this application can look properly when run in the desktop window.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;

Stage {
   title: "Colors"
   scene: Scene {
               fill: Color.web(color)
               width: space + (space + size) * num_columns
               height: space + (space + size) * num_rows
   }
}
```

The `web` function of the `Color` class enables you to create the RGB color based on the `String` color notation.

Creating Color Patterns

Now create boxes that will serve as color patterns. Boxes are allocated in matrix of 10 columns and 14 rows.

1. Add an import statement for the `Rectangle` class.

2. Add the following code fragment for the `content` instance variable of the `Scene` object:

```
          ...
Scene{
        content: for (currentColor in colors)
        Rectangle {
        def j: Integer = indexof currentColor / num_columns;
        def i: Integer = indexof currentColor - j * num_columns;
        fill: Color.web(currentColor)
        x: space + (space + size) * i
        y: space + (space + size) * j
        width: size
        height: size
        }
}
        ...
```

Note, the `currentColor` variable exists only within the cycle construction. It is used to store the current element of the `colors` sequence. Constants `i` and `j` are aimed to define location of the particular box in the matrix of the patterns. The following figure shows the values of the `Rectangle` variables for the coral color.

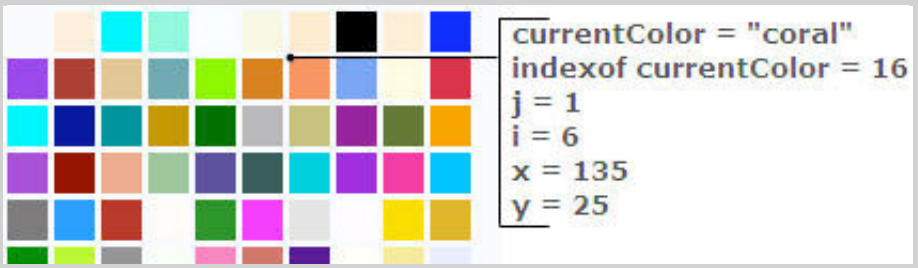

Figure 2: The `Rectangle` values for the coral color

Applying the Binding Mechanism

After creating the color patterns you can apply the binding mechanism to fill the scene with the color selected from the palette. First, add processing of the mouse events to the code of the application, and then apply the `bind` keyword to the `fill` instance variable of the `Scene` object.

1. Add the following code for the `onMouseClicked` function of the rectangle:

```
Rectangle{
        ...
        onMouseClicked: function(event) {
                color = currentColor
        }
}
```

   Refer to the Bringing Interactivity to GUI Elements for more information about handling mouse events.

2. Substitute the following code fragment for the `fill` instance variable of the `Scene` object:

```
fill: bind Color.web(color)
```

Each time a color pattern is selected, the binding relationship updates the color of the scene. The complete code of the application looks as follows. Compile, run, and test the demo both in a desktop window and on the mobile emulator.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;

def size = 19;
def space = 3;

def num_columns = 10;
def num_rows = 14;

def colors = [
  "aliceblue",       "antiquewhite",     "aqua",                "aquamarine",
  "azure",           "beige",            "bisque",              "black",
  "blanchedalmond",  "blue",             "blueviolet",          "brown",
  "burlywood",       "cadetblue",        "chartreuse",          "chocolate",
  "coral",           "cornflowerblue",   "cornsilk",            "crimson",
  "cyan",            "darkblue",         "darkcyan",            "darkgoldenrod",
  "darkgreen",       "darkgrey",         "darkkhaki",           "darkmagenta",
  "darkolivegreen",  "darkorange",       "darkorchid",          "darkred",
  "darksalmon",      "darkseagreen",     "darkslateblue",       "darkslategrey",
  "darkturquoise",   "darkviolet",       "deeppink",            "deepskyblue",
  "dimgrey",         "dodgerblue",       "firebrick",           "floralwhite",
  "forestgreen",     "fuchsia",          "gainsboro",           "ghostwhite",
  "gold",            "goldenrod",        "green",               "greenyellow",
  "grey",            "honeydew",         "hotpink",             "indianred",
  "indigo",          "ivory",            "khaki",               "lavender",
  "lavenderblush",   "lawngreen",        "lemonchiffon",        "lightblue",
  "lightcoral",      "lightcyan",        "lightgoldenrodyellow", "lightgreen",
  "lightgrey",       "lightpink",        "lightsalmon",         "lightseagreen",
  "lightskyblue",    "lightslategrey",   "lightsteelblue",      "lightyellow",
  "lime",            "limegreen",        "linen",               "magenta",
  "maroon",          "mediumaquamarine", "mediumblue",          "mediumorchid",
  "mediumpurple",    "mediumseagreen",   "mediumslateblue",     "mediumspringgreen"
  "mediumturquoise", "mediumvioletred",  "midnightblue",        "mintcream",
  "mistyrose",       "moccasin",         "navajowhite",         "navy",
  "oldlace",         "olive",            "olivedrab",           "orange",
  "orangered",       "orchid",           "palegoldenrod",       "palegreen",
  "paleturquoise",   "palevioletred",    "papayawhip",          "peachpuff",
  "peru",            "pink",             "plum",                "powderblue",
  "purple",          "red",              "rosybrown",           "royalblue",
  "saddlebrown",     "salmon",           "sandybrown",          "seagreen",
  "seashell",        "sienna",           "silver",              "skyblue",
  "slateblue",       "slategrey",        "snow",                "springgreen",
  "steelblue",       "tan",              "teal",                "thistle",
  "tomato",          "turquoise",        "violet",              "wheat",
  "white",           "whitesmoke",       "yellow",              "yellowgreen"
];

var color = "white";

Stage {
   title: "Colors"
   scene: Scene {
            fill: bind Color.web(color);
            width: space + (space + size) * num_columns
            height: space + (space + size) * num_rows
```

```
            content: for (currentColor in colors)
               Rectangle {
                 def j: Integer = indexof currentColor / num_columns;
                 def i: Integer = indexof currentColor - j * num_columns;
                 fill: Color.web(currentColor)
                 x: space + (space + size) * i
                 y: space + (space + size) * j
                 width: size
                 height: size
                 onMouseClicked: function(event) {
                 color = currentColor
                 }//OnMouseClicked
               }//Rectangle
   }//Scene
}//Stage
```

Desktop profile

- Creating an Application Window
- Adding a Circle
- Adding a Fill Pattern to the Circle
- Adding a Slider
- Defining a Binding Relationship

The goal of this application procedure is to draw a slider and a circle whose center is bound to the slider's value. The interior of the circle is painted with a radial gradient pattern. As you move the slider, the circle moves.

This example employs the slider component that resides in the `javafx.ext.swing` package, which is desktop-specific. Thus, this demo is based on the Desktop profile and will not run on the mobile devices.

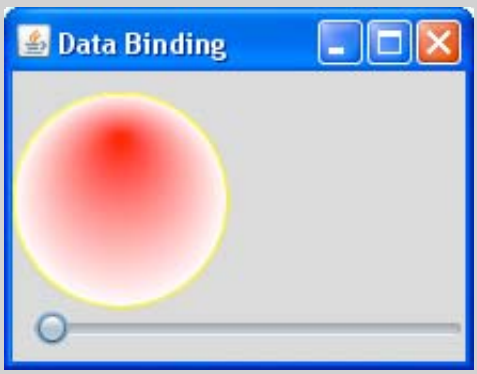

Figure 3: The data binding example after startup

The motionless gradient filling gives the circle the impression of the phases of an eclipse.

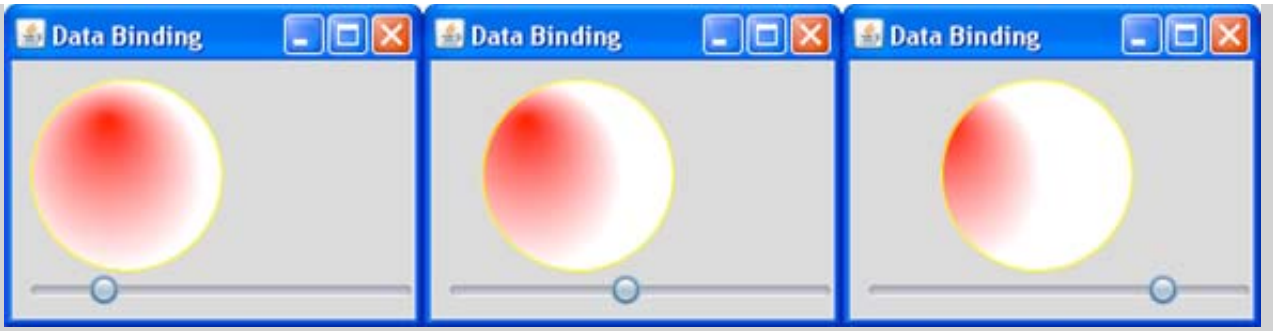

Figure 4: The position of the circle changes depending on the position of the slider

Creating an Application Window

Create an application window with the title "Data Binding", a width of 220 pixels and a height of 170 pixels. For details, see Using Declarative Syntax. The following code creates the window.

```
import javafx.stage.Stage;
import javafx.scene.Scene;

Stage {
    title: "Data Binding"
        scene: Scene {
             width: 220
             height: 170
    }//Scene
}//Stage
```

Adding a Circle

You add objects to your application window by putting them on the `scene` of the `Stage`. For more details, see Presenting UI Objects in a Graphical Scene. For details on the `Circle` class, see Using Declarative Syntax.

1. Add import statements for the `javafx.scene.shape.Circle` and `javafx.scene.paint.Color` classes.
2. Define the `content` variable of the scene by adding the `Circle` object literal.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;

Stage {
        title: "Data Binding"
        scene: Scene {
                width: 220
                height: 170
                content: Circle {
                        centerX: 50
                        centerY: 60
                        radius: 50
                        stroke: Color.YELLOW
        }//Circle
    }//Scene
}//Stage
```

By default, the interior of the circle is black and the background of the scene is white. At this step the output appears as follows.

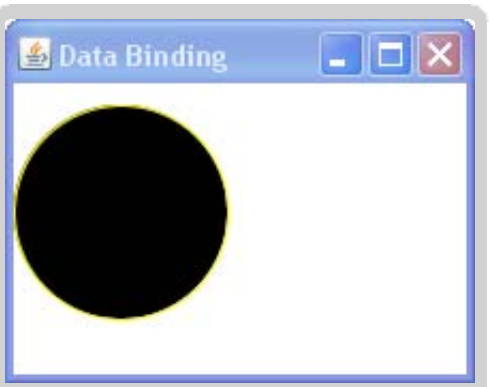

Figure 5: A circle added to the scene

3. Set the value of the `fill` variable to change the background color.

```
scene: Scene {
   fill: Color.LIGHTGRAY
   ...
}//Scene
```

Adding a Fill Pattern to the Circle

You can increase the visual attractiveness of the example by adding a specific fill pattern to the circle called radial gradient. Using the radial gradient is what gives the effect of an eclipse when the slider is moved.

To implement this painting feature, use the `RadialGradient` class. You can specify two or more colors by using the `Stop` class. The `RadialGradient` class will provide an interpolation between them. Specify a circle which controls the gradient pattern by defining a center point and a radius. You can also define a focus point within the circle where the first color is applied. The last color is applied to the perimeter of the circle.

For the radial gradient fill pattern, you can specify absolute values for the center, the radius, and the focus. In this case you need to set the `proportional` variable to `false`. If the `proportional` variable is set to `true`, values for the center, radius, and focus will range from 0.0 to 1.0, and the radial gradient will be scaled to fill the shape it is applied to.

The `stops` variable defines a sequence of colors for the radial gradient fill pattern. Use square brackets to specify a sequence, and commas to separate its elements. To add a fill pattern to the circle:

1. Add import statements for the `RadialGradient` and `Stop` classes from the `javafx.scene.paint` package.
2. Define the `fill` instance variable using the `RadialGradient` object literal.
3. Specify absolute values for a center point and a radius using the `centerX`, `centerY`, and `radius` variables.
4. Specify the focus point by using the `focusX` and `focusY` variables.
5. Set the `proportional` variable to `false`.
6. Define the `stops` variable as a sequence of red and white colors.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
```

```
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
import javafx.scene.paint.RadialGradient;
import javafx.scene.paint.Stop;

Stage {
    title: "Data Binding"
    scene: Scene {
        fill: Color.LIGHTGRAY
        width: 220
        height: 170
        content: Circle {
            centerX: 50 centerY : 60 radius: 50
            stroke: Color.YELLOW
            fill: RadialGradient {
                centerX: 50 centerY : 60 radius: 50
                focusX: 50 focusY: 30
                proportional: false
                stops: [
                    Stop {offset: 0 color: Color.RED},
                    Stop {offset: 1 color: Color.WHITE}
                ]
            }//RadialGradient
        }//Circle
    }//Scene
}//Stage
```

Compiling and running this modified code produces the following result.

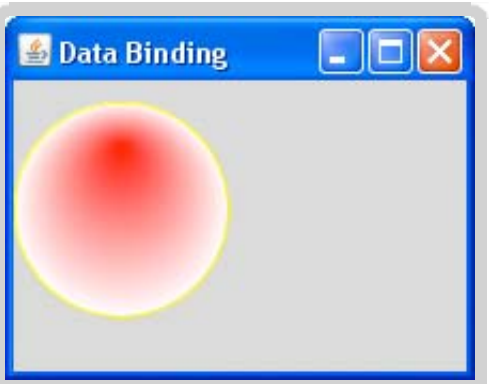

Figure 6: The circle filled in with the radial gradient pattern

Adding a Slider

The `SwingSlider` class from the `javafx.ext.swing` package provides a slider control, a widely known desktop GUI element. Add the slider to the scene and place it below the circle. By default, the slider will be rendered at the point (0,0). You can use the `translateX` and `translateY` instance variables to specify distances by which coordinates are translated in the X axis and Y axis directions on the scene. To add a slider, complete the following steps.

1. Add an import statement for the `javafx.ext.swing.SwingSlider` class.
2. A dd the `SwingSlider` object literal to the `content` variable of the scene.
3. Specify the slider's minimum and maximum values and set the current value to zero.

```
import javafx.ext.swing.SwingSlider;

content: [
    Circle {
       ...
    },
    SwingSlider {minimum: 0 maximum: 60 value : 0}
]
```

4. Specify the position of the slider within the scene using the `translateX` and `translateY` instance variables.

```
content: [
    Circle {
       ...
    },
    SwingSlider {
        minimum: 0
        maximum: 60
        value : 0
        translateX: 10
        translateY: 110
    }
]
```

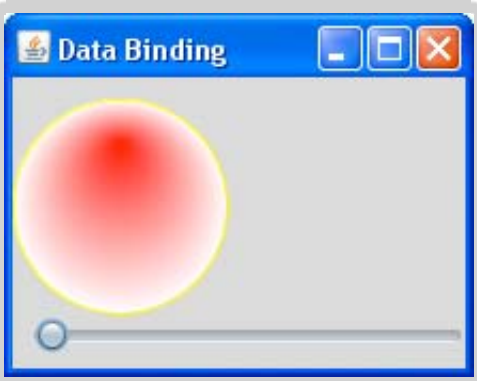

Figure 7: A slider added to the application window

Defining a Binding Relationship

Finally, bind the center of the circle to the slider's value. The slider is displayed in the application window and you can change its value by moving the knob. However, you have no means to refer to the slider's value, which is required for binding. The solution is to define a `slider` variable and then refer to the `slider.value`. To define a binding relationship, complete the following steps.

1. Create the `slider` variable.

```
var slider = SwingSlider{minimum: 0 maximum: 60 value: 0 translateX: 10 translateY: 1
```

2. Define the binding relationship.

```
Circle {
    centerX: bind slider.value+50 centerY: 60 radius: 50
  ...
```

The `bind` operator keeps track of any changes in the value on the right. As the slider's value changes, the center of the circle is updated and JavaFX automatically renders the circle at the new location. Since the position of the radial gradient filling does not change, you can see how the circle shifts relative to the initial filling.

In conclusion, the complete code of the data binding example is shown in the following image. Compile, run and explore data binding in action.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.ext.swing.SwingSlider;

import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
import javafx.scene.paint.Stop;
import javafx.scene.paint.RadialGradient;

var slider = SwingSlider{minimum: 0 maximum: 60 value: 0 translateX: 10 translateY: 1

Stage {
    title: "Data Binding"
    width: 220
    height: 170
    scene: Scene {
        fill: Color.LIGHTGRAY;
        content: [
            slider,
            Circle {
                centerX: bind slider.value+50 centerY: 60 radius: 50
                    stroke: Color.YELLOW
                    fill: RadialGradient {
                        centerX: 50 centerY: 60 radius: 50
                        focusX: 50 focusY: 30
                        proportional: false
                        stops: [
                            Stop {offset: 0 color: Color.RED},
                            Stop {offset: 1 color: Color.WHITE},
                        ]
                }//RadialGradient
            }//Circle
      ]
  }//Scene
}//Stage
```

Conclusion

In this lesson you learned how to create a data binding relationship. Use the data binding mechanism whenever you want to update one variable in response to another.

http://java.sun.com/javafx/1/tutorials/ui/layout/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 6: Laying Out GUI Elements

Download tutorial

The layout mechanism in the JavaFX SDK enables you to easily arrange and align components without specifying absolute coordinates for each UI object. Although absolute coordinates provide a certain flexibility, there are cases when you may find the layout mechanism more convenient. The example code provided in this tutorial uses data binding and the declarative syntax of the JavaFX Script programming language. Refer to Learning the JavaFX Script Programming Language for more details on these concepts.

This lesson introduces two examples. The first example demonstrates how the layout mechanism can be applied to the common profile of the JavaFX API, which includes use in desktop and mobile applications. The second example shows how to arrange geometric shapes and standard UI components available in the `javafx.ext.Swing` package. Thus, the second application is desktop-specific.

Common profile

This application renders a simple play list consisting of two columns. The first column contains a list of songs highlighted with the color bars, the second column includes the play buttons. The application is designed only to create graphical elements and arrange them using the layout mechanism. Refer to Bringing Interactivity to GUI Elements for more information on how to handle mouse events and make this application interactive.

Figure 1: Play List

Creating an Application Scene

In Presenting UI Objects in a Graphical Scene, you learned that UI components, shapes, text, and images are considered a hierarchy of objects in a graphical scene. Perform the following steps to create an application scene:

1. Add an `import` statements for the `Stage`, `Scene` and `Color` classes.
2. Add the `Stage` object literal.
3. Specify the title of the window.
4. Add the `Scene` object literal to the `scene` instance variable of the `Stage` class.
5. Set the width and the height of the scene.
6. Define the color of the scene using the `fill` variable of the `Scene` class.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;

Stage {
    title: "Play list"
    scene:
        Scene{
            fill: Color.NAVY
            width: 250
            height: 230
        }//Scene
}//Stage
```

Creating the List of Songs

To visualize a list of five songs perform the following operations:

1. Add `import` statements for the `Text` and `Font` classes.
2. Define the `songs` sequence to store the name of the songs.
3. Create a sequence of the `Text` object within a cycle construction
4. Set the color for the text elements using the `fill` instance variable of the `Text` object
5. Set the font size to `12`

The following code fragment shows all these modifications:

```
import javafx.scene.text.Text;
import javafx.scene.text.Font;

def songs = [
  "Helter Skelter",
  "Cry Baby Cry",
  "Revolution 1",
  "Good Night",
  "Julia"
];

var playList = for (song in songs)
           Text{
                y: 20
                x: 5
                fill: Color.BLACK
                font: Font {size: 12}
                content: song
           }//Text
```

Adding Color Bars

You can visually enhance the play list by adding contrast color bars to each text item. The colors of the bars alter, so the odd text items are highlighted with light grey, and the even items are highlighted with light blue. Perform the following steps to construct the bars using the `Rectangle` shape.

1. Add `import` statements for the `Group`, `Rectangle`, `LinearGradient`, and `Stop` classes.

2. Modify the code for the `playList` variable to create a sequence of `Group` objects that contain `Text` and `Rectangle` objects. It

is important to keep the order of elements within the group so that text will be rendered over the rectangles.

```
import javafx.scene.Group;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.text.Text;
import javafx.scene.text.Font;

var playList = for (song in songs)
        Group{
             content:[
                 Rectangle{...},
                 Text{...}
             ]
        }//Group
```

3. Construct the `Rectangle` objects specifying their width and the height.

4. Use the `RadialGradient` pattern to fill the rectangles. The following expression checks whether the list item is odd or even: `if (indexof song mod 2 == 0)`. If the item is odd, the color defined by the first `Stop` object is `Color.LIGHTGREY`, otherwise, it is `Color.LIGHTBLUE`.

The following code fragment shows all these modifications:

```
import javafx.scene.Group;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.text.Text;
import javafx.scene.text.Font;

def songs = [
    "Helter Skelter",
    "Cry Baby Cry",
    "Revolution 1",
    "Good Night",
    "Julia"
];

var playList = for (song in songs)
Group{
    content:[
        Rectangle{
            width: 200
            height: 25
            fill: LinearGradient {
                startY: 5
                endX:190,
                endY: 10
                proportional: false
                stops: [
                    Stop {
                        offset: 0.0
                        color: if (indexof song mod 2 == 0)
```

```
                                then Color.LIGHTGREY
                                else Color.LIGHTBLUE
                    },
                    Stop {offset: 1.0 color: Color.NAVY}
                ]
            }//LinearGradient
        },//Rectangle
        Text{
            y: 20
            x: 5
            fill: Color.BLACK
            font: Font {size: 12}
            content: song
        }//Text
    ]
}//Group
```

Refer to Creating Graphical Objects for more information about graphic shapes and visual effects.

Creating the Play Buttons

Create a second column of elements that will contain play buttons. Perform the following steps to create a sequence of button images:

1. Add `import` statements for the `ImageView` and `Image` classes.
2. Define an image for the play button. Use the **`playButton`** image.
3. Create a sequence of the `ImageView` elements within a cycle construction

In the JavaFX SDK, images are created using the `Image` class, where the image location is specified in the `url` instance variable. Note that only a `Node` object can be added to a scene's content, so you need to use an adapter class, called `ImageView`. Refer to Presenting UI Objects in a Graphical Scene for more details about nodes. When specifying the image url, you can set the URI of the resource or use the relative codebase. In this example the image url is set using the `__ DIR__` variable that indicates the directory where the image is located. By default it points to the current directory, so make sure that the `playButton` image is located in the same directory as application compiled classes. To run the application on the mobile emulator make sure that the image is packed into the application jar file along with the compiled classes.

```
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

def image = Image{url: "{__DIR__}/playButton.jpg" }

var playButtons = for (song in songs)
        ImageView{image: image}
```

Creating the Layout

The preceding code fragments did not specify coordinates for the `Group` and `ImageView objects.` These elements will be arranged using the layout mechanism. The JavaFX SDK provides `VBox` and `HBox` classes of the `javafx.scene.layout` package to create both vertical and horizontal layout. The `VBox` class arranges components vertically, while the `HBox` class arranges them horizontally. Use a `VBox` object to create a column with the list items, another `VBox` object for the column with the play buttons, and a `HBox` object to combine these two columns.

Figure 2: A Combination of the HBox and VBox objects

Perform the following steps to apply the layout mechanism:

1. Add `import` statements for the `VBox` and `HBox` classes.
2. Add an `HBox` object literal to the `content` variable of the `Scene` object.
3. Use the `VBox` class to lay out elements of the `playList` sequence.
4. Define an interval in pixels between elements of the `VBox` object using the `spacing` instance variable.

```
VBox{spacing: 12 content: playList}
```

5. Use another `VBox` object to arrange the elements of the `playButtons` sequence.
6. Set the `spacing` instance variable to `5`.

```
VBox{spacing: 5 content: playButtons}
```

See the complete code of the example as follows.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.text.Text;
import javafx.scene.text.Font;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
```

```
import javafx.scene.layout.VBox;
import javafx.scene.layout.HBox;


def songs = [
  "Helter Skelter",
  "Cry Baby Cry",
  "Revolution 1",
  "Good Night",
  "Julia"
];

var playList = for (song in songs)
      Group{
             content:[
                 Rectangle{
                        width: 200
                        height: 25
                        fill: LinearGradient {startY: 5
                                endX:190, endY: 10 proportional: false
                                stops: [
                                   Stop {offset: 0.0 color:
                                        if (indexof song mod 2 == 0)
                                             then Color.LIGHTGREY
                                             else Color.LIGHTBLUE
                                   },
                                   Stop {offset: 1.0 color: Color.NAVY}
                                ]
                        }//LinearGradient
                  },//Rectangle
                  Text{
                        y: 20
                        x: 5
                        fill: Color.BLACK
                        font: Font {size: 12}
                        content: song
                  }//Text
             ]
        }//Group

def image = Image{url: "{__DIR__}/playButton.jpg" }

var playButtons = for (song in songs)
        ImageView{image: image}

Stage {
    title: "Play list"
    scene:
        Scene{
            fill: Color.NAVY
            width: 250
            height: 230
            content:
              HBox{
               content:[
                 VBox{spacing: 12 content: playList},
                 VBox{spacing: 5 content: playButtons}
               ]
              }//HBox
        }//Scene
}//Stage
```

When compiled and run on the mobile emulator this code produces the following output.

Figure 3: The Playlist demo run on the mobile emulator

Desktop profile

- Creating an Application Window
- Defining Radio Buttons
- Creating Circles
- Creating the Layout

Consider creating three circles and a toggle group of radio buttons. Each radio button controls a color in the corresponding circle. When the radio button is selected the color is applied, otherwise the circle is grey. This scenario simulates a traffic light and is the example used in this tutorial to describe the layout mechanism.

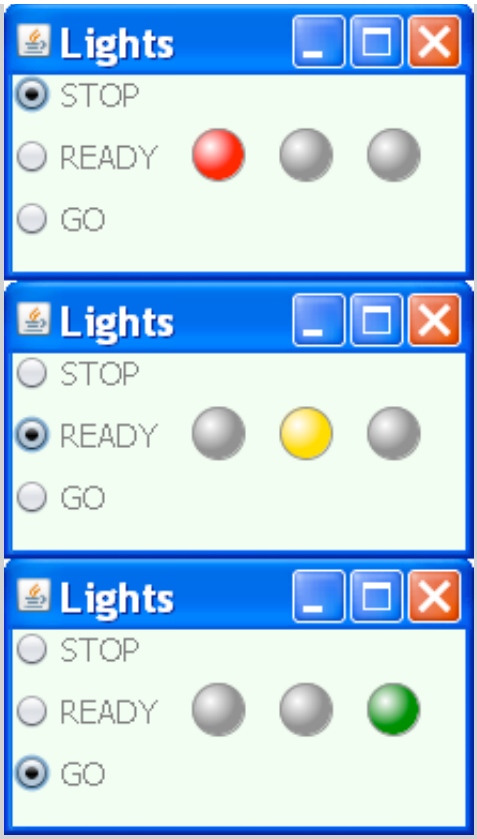

Figure 4: Traffic Lights

Creating an Application Window

To create a window perform the following steps:

1. Add an `import` statement for the `Stage`, `Scene` and `Color` classes.
2. Add the `Stage` object literal.
3. Specify the title of the window.
4. Add the `Scene` object literal to the `scene` instance variable of the `Stage` class.
5. Set the width and the height of the scene.
6. Define the color of the scene using the `fill` variable of the `Scene` class.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
Stage {
   title: "Lights"
   scene:
     Scene{
      width: 220
      height: 130
      fill: Color.HONEYDEW
         }//Scene
}//Stage
```

Defining Radio Buttons

Create a group of radio buttons in which only one button can be selected at a given time. This is called a *toggle group*. Perform the

following steps:

1. Add `import` statements for the `SwingToggleGroup`, `SwingRadioButton`, and `Font` classes.
2. Define the toggle group using the `SwingToggleGroup` class.

> Note: In the example code for this lesson, you will define a variable name for each UI element so that you can easily learn how the layout mechanism works. You will also apply binding to the UI elements by referring to the variable names.

3. Define the `choiceText` sequence for the button captions.
4. Create a sequence of radio buttons within a cycle construction. The buttons enable selection of the particular traffic light using the `SwingRadioButton` class.
5. Add each radio button to the group using the `toggleGroup` instance variable.
6. Use the `font` instance variable in order to alter the caption style of the radio buttons.

The following code fragment defines a toggle group, then creates radio buttons adding them to the group.

```
import javafx.ext.swing.SwingToggleGroup;
import javafx.ext.swing.SwingRadioButton;
import javafx.scene.text.Font;


var group = SwingToggleGroup{};
var choiceText = ["STOP", "READY", "GO"];

var choices =  for (text in choiceText)
SwingRadioButton{
           text: text
           foreground: Color.GRAY
           font: Font{name:"Tahoma" size: 15}
           toggleGroup: group

}//SwingRadioButton
```

Creating Circles

To draw circles that indicate the traffic lights:

1. Add `import` statements for the `Circle`, `Color`, `RadialGradient`, and `Stop` classes.
2. Define the `colors` sequence for the colors of the circles.
3. Create a sequence of circles within a cycle construction.
4. Apply radial gradient filling to visually enhance the example.

```
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
import javafx.scene.paint.RadialGradient;
import javafx.scene.paint.Stop;

var colors = ["RED", "GOLD", "GREEN"];

var lights = for (color in colors)
 Circle {
     centerX: 12
     centerY: 12
     radius: 12
```

```
    stroke: Color.GRAY
    fill: bind RadialGradient {
            centerX: 8,
            centerY: 8,
            radius: 12,
            proportional: false
            stops: [
                   Stop {offset: 0.0 color: Color.WHITE},
                   Stop {offset: 1.0 color:
                               if (choices[indexof color].selected)
                                      then Color.web(color)
                                      else Color.GREY
                   }//Stop
            ]
     }//RadialGradient
}//Circle
```

The preceding code sample uses the data binding mechanism to change the color of the circle. If the `selected` instance variable of `choices[1]` is `true`, the `color` instance variable becomes `Color.RED`, otherwise it is `Color.GREY`. Refer to Creating Graphical Objects for more information about shapes and visual effects.

Creating the Layout

Once all components are created, you can use the `HBox` and `VBox` classes to lay them out in a scene. In this example you need a `VBox` object to layout the radio buttons, a `HBox` object for the circles, and another `HBox` to arrange those two boxes.

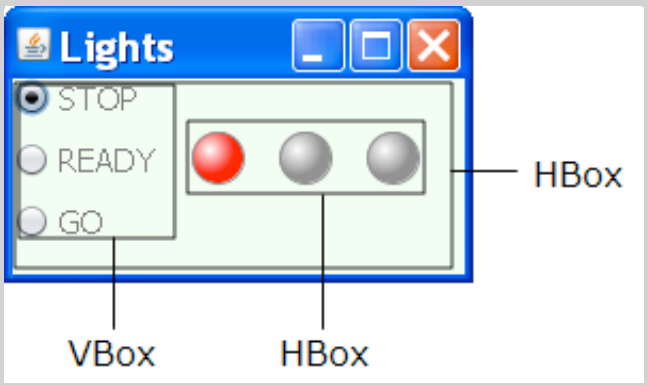

Figure 5: A combination of HBox and VBox

Perform the following steps :

1. Add the corresponding import statements for the `VBox` and `HBox` classes.
2. Arrange the radio buttons within the vertical box using the following code line:

```
VBox{content: choices}
```

3. Define an interval in pixels between elements of the `VBox` object using the `spacing` instance variable.
4. Add the circles to the `HBox` object's content using the following code line.

```
HBox{spacing: 15 content: lights}
```

5. Set the `translateY` instance variable to `25` to specify a vertically offset for the horizontal box.

6. Use another `HBox` object to layout the vertical box with the radio buttons and the horizontal box with the circles.

See the complete code of the example as follows.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.ext.swing.SwingToggleGroup;
import javafx.ext.swing.SwingRadioButton;
import javafx.scene.text.Font;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;
import javafx.scene.paint.RadialGradient;
import javafx.scene.paint.Stop;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;

var group = SwingToggleGroup{};
var choiceText = ["STOP", "READY", "GO"];
var colors = ["RED", "GOLD", "GREEN"];

var choices = for (text in choiceText)
SwingRadioButton{
    text: text
    foreground: Color.GRAY
    font: Font{name:"Tahoma" size: 15}
    toggleGroup: group
}//SwingRadioButton

var lights = for (color in colors)
Circle {
    centerX: 12
    centerY: 12
    radius: 12
    stroke: Color.GRAY
    fill: bind RadialGradient {
        centerX: 8,
        centerY: 8,
        radius: 12,
        proportional: false
        stops: [
            Stop {offset: 0.0 color: Color.WHITE},
            Stop {offset: 1.0 color:
                    if (choices[indexof color].selected)
                          then Color.web(color)
                          else Color.GREY
            }//Stop
        ]
    }//RadialGradient
}//Circle

Stage {
    title: "Lights"
    scene: Scene{
                width: 220
            height: 130
```

```
            fill: Color.HONEYDEW
            content: HBox{spacing: 10 content:[
                       VBox{ spacing: 10 content: choices},
                       HBox{ spacing: 15 content: lights translateY: 25}
                     ]
            }//HBox
     } //Scene
}//Stage
```

When compiled and run this code produces the following window.

Figure 6: A vertical box with radio buttons and a horizontal box with circles

Conclusion

In this lesson you learned how to layout UI components in a scene. You can use the layout mechanism, absolute coordinates, or a combination of those two approaches for your current task. Refer to the following articles for more information about dynamic and group layout: Layout Primer for JavaFX 1.0, How to lay out FX nodes.

http://java.sun.com/javafx/1/tutorials/ui/animation/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 7: Creating Animated Objects

Download tutorial

The JavaFX libraries provide built-in support to create animation. This lesson shows you how to build a graphical object and then animate it using linear interpolation, a type of key frame animation supported by JavaFX libraries. The example introduced in this lesson uses the declarative syntax of the JavaFX Script Language, as well as data binding, graphics, and node-specific features, so it may be helpful for you to become familiar with Learning the JavaFX Script Programming Language, Presenting UI Objects in a Graphical Scene, and Creating Graphical Objects.

This lesson is divided into two sections. The common profile includes actions for using animation both for mobile and desktop applications. The second section adds some visual effects to run the application in the desktop window.

Common Profile

This section introduces a step-by-step procedure that demonstrates how to add animation to a simple application. It will guide you through the creation of a cloud that travels across a sky of sunshine, and bounces off the window borders, as displayed in the following window.

Figure 1: Cloud bouncing within window

Creating an Application Scene

As you learned in Presenting UI Objects in a Graphical Scene, UI components, shapes, text, and images are considered a hierarchy of objects in a graphical scene. Animation of these graphical objects also takes place in scene, so the first step is to create a scene.

1. Add `import` statements for the `Stage`, `Scene`, and `Color` classes.
2. Add the `Stage` object literal and define the `title` instance variable.
3. Add the `Scene` object literal to the `scene` instance variable of the `Stage` class.
4. Define the color of the scene using the `fill` variable of the `Scene` class.

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;

Stage{
  title: "Cloud"
  scene:
     Scene{
       fill: Color.WHITE
     }//Scene
}//Stage
```

Refer to Using Declarative Syntax for more information about declarative syntax employed in the JavaFX Script programming language.

Adding a Background Image

In the JavaFX SDK, images are created using the `javafx.scene.image.Image` class, where the image location is specified in the `url` instance variable. Note that only a `Node` object can be added to a scene's content, so you need to use an adapter class, called `ImageView`. More details about scene and nodes are in Presenting UI Objects in a Graphical Scene.

1. Add two new imports for the `Image` and `ImageView` classes.
2. Set an image that will serve as a background for the traveling cloud. Use the sun.jpg image. When specifying the image url, you can set the URI of the resource or use the relative codebase. In this example the image url is set using the `__ DIR__` variable that indicates the directory where the image is located. By default it points to the current directory, so make sure that the `sun` image is located in the same directory as application compiled classes. To run the application on the mobile emulator make sure that the image is packed into the application jar file along with the compiled classes.

These changes are reflected in the modified code shown below:

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
Stage{
  title: "Cloud"
  scene:
     Scene{
       fill: Color.WHITE
       content:[
        ImageView{
                    image: Image{
                               url:  "{__DIR__}sun.jpg"
                     }//Image
        }//ImageView
       ]
```

```
    }//Scene
}//Stage
```

When compiled and run on the mobile emulator, this modified code produces the following window.

Figure 2: A scene with an image as background

Drawing a Cloud

Create the actual cloud by drawing five successive arcs, joining the last one to the first. The end point of the first arc is the start point of the second arc. This is illustrated in the following diagram.

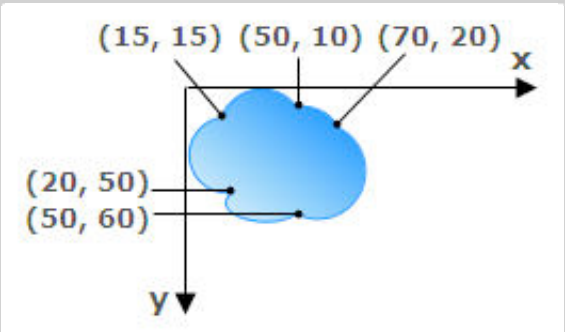

Figure 3: End-points of arcs that form cloud

To draw this cloud in your code you need to perform the following steps:

1. Add `import` statements for the `MoveTo`, `ArcTo`, `Path`, `LinearGradient`, and `Stop` classes. Refer to Creating Graphical Objects for more information about shapes and filling methods.

2. Use the `MoveTo`, `ArcTo`, and `Path` classes from the `javafx.scene.shape` package as shown in the following code fragment.

```
   Path {
       stroke: Color.DODGERBLUE
       fill: LinearGradient {
           startX:60,
           startY:10,
           endX:10
           endY:80 ,
           proportional: false
           stops: [
               Stop {offset: 0.0 color: Color.DODGERBLUE},
               Stop {offset: 0.5 color: Color.LIGHTSKYBLUE},
               Stop {offset: 1.0 color: Color.WHITE}
           ]
       }
       elements: [
           MoveTo {x: 15 y: 15 },
           ArcTo {x: 50 y: 10 radiusX: 20 radiusY: 20 sweepFlag: true},
           ArcTo {x: 70 y: 20 radiusX: 20 radiusY: 20 sweepFlag: true},
           ArcTo {x: 50 y: 60 radiusX: 20 radiusY: 20 sweepFlag: true},
           ArcTo {x: 20 y: 50 radiusX: 10 radiusY: 5 sweepFlag: true},
           ArcTo {x: 15 y: 15 radiusX: 10 radiusY: 10 sweepFlag: true},
        ]
    }//Path
```

The `MoveTo` class indicates the start point for the shape, and the `ArcTo` class creates an arc segment. All segments are combined into a shape using the `Path` class which represents a simple shape, and enables basic construction of a geometric path. The `Path` class is helpful when you need to create your own shape that is different from the primitive graphic shapes available in the `javafx.scene.shape` package. The `Path` class extends the `Node` class and inherits all of its instance variables and functions.

Note: The `sweepFlag` instance variable is set to `true` so that the arc be drawn clockwise, in a "positive" angle. If the arcs are drawn counterclockwise, they will not curve correctly.

The following modified code includes a graphical scene, an image, and a cloud:

```
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.shape.ArcTo;
import javafx.scene.shape.MoveTo;
import javafx.scene.shape.Path;

Stage{
    title: "Cloud"
    scene: Scene{
            fill: Color.WHITE
            content:[
                ImageView{
                    image: Image{
                        url: "{__DIR__}sun.jpg"
```

```
                    }
                },//ImageView
                Path {
                    stroke: Color.DODGERBLUE
                    fill: LinearGradient {
                        startX:60,
                        startY:10,
                        endX:10
                        endY:80 ,
                        proportional: false
                        stops: [
                            Stop {offset: 0.0 color: Color.DODGERBLUE},
                            Stop {offset: 0.5 color: Color.LIGHTSKYBLUE},
                            Stop {offset: 1.0 color: Color.WHITE}
                        ]
                    }//LinearGradient
                    elements: [
                        MoveTo {x: 15 y: 15 },
                        ArcTo {x: 50 y: 10 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 70 y: 20 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 50 y: 60 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 20 y: 50 radiusX: 10 radiusY: 5 sweepFlag: true},
                        ArcTo {x: 15 y: 15 radiusX: 10 radiusY: 10 sweepFlag: true},
                    ]
                }//Path
            ]
        }//Scene
}//Stage
```

When compiled and run in a desktop window, this modified code produces the following window.

Figure 4: A window with an image and a cloud-looking shape

Creating Horizontal Motion

The next step is to animate the cloud. The JavaFX Script Language supports the *Key Frame animation* concept. This means that the animated state transitions of the graphical scene are declared by start and end snapshots (key frames) of the scene's state at certain points in time. Given these two states, the system can automatically perform the animation. It can stop, pause, resume, reverse or repeat movement when requested.

First, simplify the task by animating the cloud horizontally, with no vertical motion. Later you will add in the vertical motion. To animate the could horizontally, alter the `translateX` instance variable of the `Path` object, and leave the `translateY` instance variable constant. Perform the following steps:

1. S et the `translateY` variable of the `Path` object to 100.
2. Define key frames for the start point (0, 100) and the end point (158, 100). To calculate these values, take into consideration the image size, which is 241x332, and the shape size, which is 83x64. These measurements are illustrated in the following diagram.

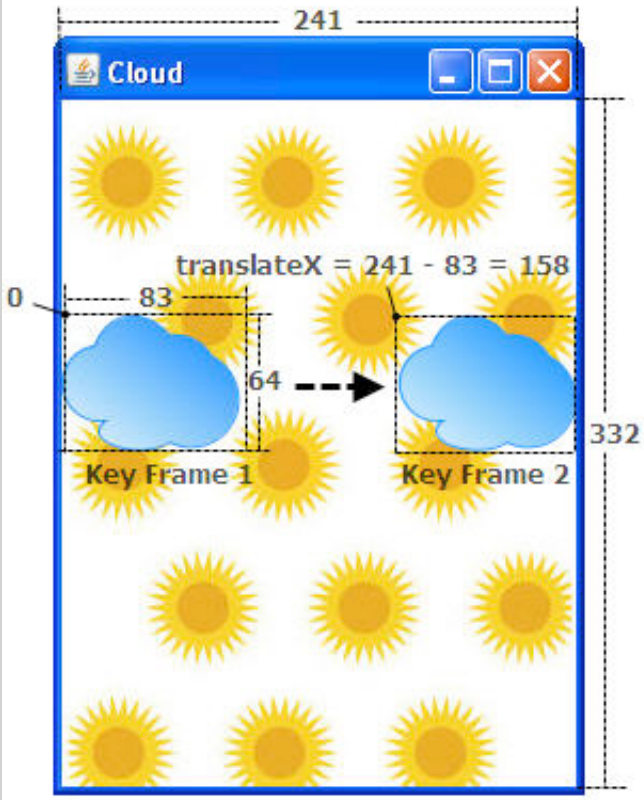

Figure 5: Key Frames

Animation occurs along a *timeline*, represented by a `javafx.animation.Timeline` object. Each timeline contains two or more key frames, represented by `javafx.animation.KeyFrame` objects.

3. Create a timeline with two key frames to perform the cloud's horizontal movement and starts the movement when the application is launched. Positions between the start and the end points are calculated using linear interpolation.

```
import javafx.animation.Timeline;
import javafx.animation.KeyFrame;
import javafx.animation.Interpolator;
```

```
var x: Number;

Timeline {
  keyFrames: [
         KeyFrame{
              time: 0s
              values: x => 0.0
         },

         KeyFrame{
              time: 4s
              values: x => 158.0 tween Interpolator.LINEAR
         }
  ]
}.play();
```

The `time` instance variable defines the elapsed time at which the associated values will be set within a single cycle of the `Timeline` object. The `time` is a variable of the `javafx.lang.Duration` class that takes a `Number` value followed by "s" or "ms" to indicate seconds or milliseconds. The `=>` operator provides a literal constructor for a list of key values. The `tween` operator is a literal constructor for an interpolated value. Therefore the cloud begins at pixel 0 and moves to position 158 over the course of four seconds.

Although `KeyFrame` animations are typical JavaFX objects, special syntax is provided to make it easier to express animation than is possible with the standard object-literal syntax. The trigger clause enables you to associate an arbitrary callback with the key frame. The time specified by `at` is relative to the start of the timeline. This capability simplifies the code as follows:

```
import javafx.animation.Timeline;
import javafx.animation.Interpolator;

var x: Number;

Timeline {
  keyFrames: [
         at (0s) {x => 0.0},
         at (4s) {x => 158.0 tween Interpolator.LINEAR}
  ]
}.play();
```

4. Bind the `translateX` instance variable of the `Path` object to the `x` variable as shown in the following code fragment:

```
Path{
  ...
  translateX: bind x
  ...
}
```

When the `x` variable changes, the `translateX` variable of the `Path` object also changes. More details about the Data Binding mechanism are in Applying Data Binding to UI Objects.

Controlling the Timeline Cycle

You can use instance variables of the `Timeline` class to control the timeline cycle.

1. Set the `repeatCount` instance variable to `Timeline.INDEFINITE` to loop the animation.
2. Set the `autoReverse` instance variable to `true` to enable two-way movement.

The following code accomplishes these tasks:

```
import javafx.animation.Timeline;
import javafx.animation.Interpolator;

var x: Number;

Timeline {
  repeatCount: Timeline.INDEFINITE
  autoReverse: true
  keyFrames: [
         at (0s) {x => 0.0},
         at (4s) {x => 158.0 tween Interpolator.LINEAR}
  ]
}.play();
```

The modified code of the application is shown below:

```
import javafx.animation.Interpolator;
import javafx.animation.Timeline;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.shape.ArcTo;
import javafx.scene.shape.MoveTo;
import javafx.scene.shape.Path;

var x: Number;

Timeline {
  repeatCount: Timeline.INDEFINITE
  autoReverse: true
  keyFrames: [
   at (0s) {x => 0.0},
   at (4s) {x => 158.0 tween Interpolator.LINEAR}
  ]
}.play();

Stage{
    title: "Cloud"
    scene: Scene{
            fill: Color.WHITE
            content:[
                ImageView{
                    image: Image{
                        url: "{__DIR__}sun.jpg"
                    }},
                Path {
                    translateX: bind x
```

```
                    translateY: 100
                    stroke: Color.DODGERBLUE
                    fill: LinearGradient {
                        startX:60,
                        startY:10,
                        endX:10
                        endY:80 ,
                        proportional: false
                        stops: [
                            Stop {offset: 0.0 color: Color.DODGERBLUE},
                            Stop {offset: 0.5 color: Color.LIGHTSKYBLUE},
                            Stop {offset: 1.0 color: Color.WHITE}
                        ]
                    }
                    elements: [
                        MoveTo {x: 15 y: 15 },
                        ArcTo {x: 50 y: 10 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 70 y: 20 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 50 y: 60 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 20 y: 50 radiusX: 10 radiusY: 5 sweepFlag: true},
                        ArcTo {x: 15 y: 15 radiusX: 10 radiusY: 10 sweepFlag: true},
                    ]
                }//Path
            ]
        }//Scene
}//Stage
```

When compiled and run this code produces the following window:

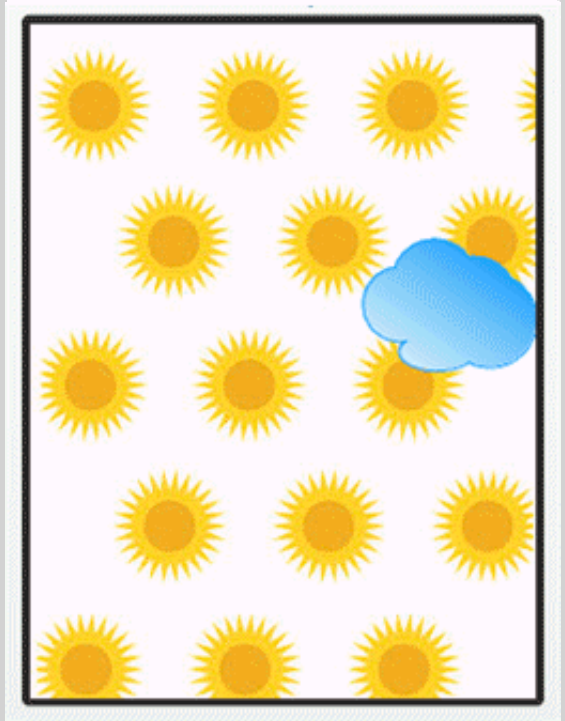

Figure 6: Horizontal movement

This animation application uses linear interpolation which moves the object in even time increments. You can play with other forms of interpolation. For example, if you set the `Interpolator.EASEBOTH` type, the cloud will slightly slow down at the start and at the end of

the timeline cycle.

Adding Vertical Motion

You can enhance the application by creating the originally desired floating effect.

1. Create another timeline for the `y` coordinate of the shape.
2. Bind the `translateY` instance variable to the `y` value as shown on the following code fragment:

```
var y: Number;

Timeline {
  repeatCount: Timeline.INDEFINITE
  autoReverse: true
  keyFrames: [
    at (0s) {y => 0.0},
    at (7s) {y => 258.0 tween Interpolator.LINEAR},
  ]
}.play();
...

Path{
   ...
   translateY: bind y
   ...
}//Path
```

> Note: The `y` variable attains its maximum position after seven seconds, which is faster than the `x` variable. Therefore, the `translateY` value changes faster than `translateX`. This produces a wandering effect.

The following is the complete code of the example.

```
import javafx.animation.Interpolator;
import javafx.animation.Timeline;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.paint.LinearGradient;
import javafx.scene.paint.Stop;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.shape.ArcTo;
import javafx.scene.shape.MoveTo;
import javafx.scene.shape.Path;

var x: Number;

Timeline {
  repeatCount: Timeline.INDEFINITE
  autoReverse: true
  keyFrames: [
   at (0s) {x => 0.0},
   at (4s) {x => 158.0 tween Interpolator.LINEAR}
  ]
```

```
}.play();

var y: Number;

Timeline {
  repeatCount: Timeline.INDEFINITE
  autoReverse: true
  keyFrames: [
    at (0s) {y => 0.0},
    at (7s) {y => 258.0 tween Interpolator.LINEAR},
  ]
}.play();

Stage{
    title: "Cloud"
    scene: Scene{
            fill: Color.WHITE
            content:[
                ImageView{
                    image: Image{
                        url: "{__DIR__}sun.jpg"
                    }},
                Path {
                    translateX: bind x
                    translateY: bind y
                    stroke: Color.DODGERBLUE
                    fill: LinearGradient {
                        startX:60,
                        startY:10,
                        endX:10
                        endY:80 ,
                        proportional: false
                        stops: [
                            Stop {offset: 0.0 color: Color.DODGERBLUE},
                            Stop {offset: 0.5 color: Color.LIGHTSKYBLUE},
                            Stop {offset: 1.0 color: Color.WHITE}
                        ]
                    }
                    elements: [
                        MoveTo {x: 15 y: 15 },
                        ArcTo {x: 50 y: 10 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 70 y: 20 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 50 y: 60 radiusX: 20 radiusY: 20 sweepFlag: true},
                        ArcTo {x: 20 y: 50 radiusX: 10 radiusY: 5 sweepFlag: true},
                        ArcTo {x: 15 y: 15 radiusX: 10 radiusY: 10 sweepFlag: true},
                    ]
                }//Path
            ]
     }//Scene
}//Stage
```

When compiled and run, this code produces the following window.

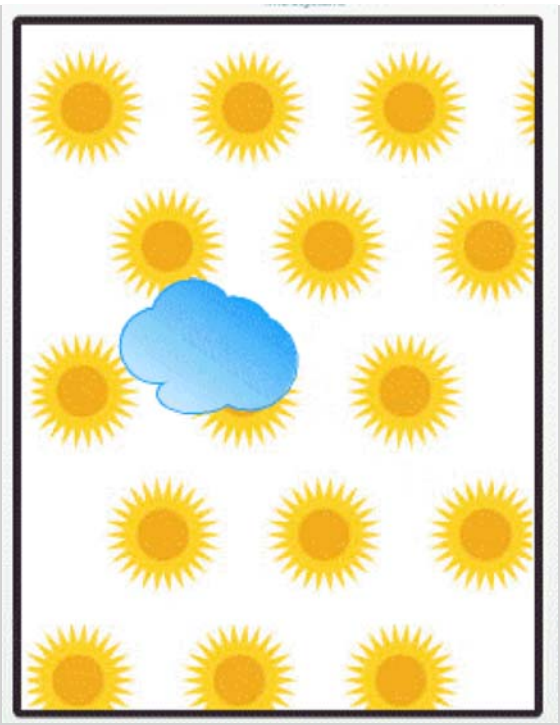

Figure 7: Cloud bouncing within window

Desktop Profile

You can apply one of the effects available in the `javafx.scene.effect` and `javafx.scene.effect.light` packages to visually enhance the application. These packages exist only in the desktop profile. Perform the following steps to create the lighting effect and make the cloud seem embossed.

1. Add import statements for the `javafx.scene.effect.Lighting` and `javafx.scene.effect.light.DistantLight` classes
2. Apply the following code for the `effect` instance variable of the `Path` object.

```
effect: Lighting{light: DistantLight{azimuth: 90}}
```

This effect simulates lighting up the object with a distant light source. The azimuth instance variable defines the angle of the light source.

The complete code of the desktop-specific application is located in the **`cloudDesktop.fx`** file. When compiled and run this code produces the window.

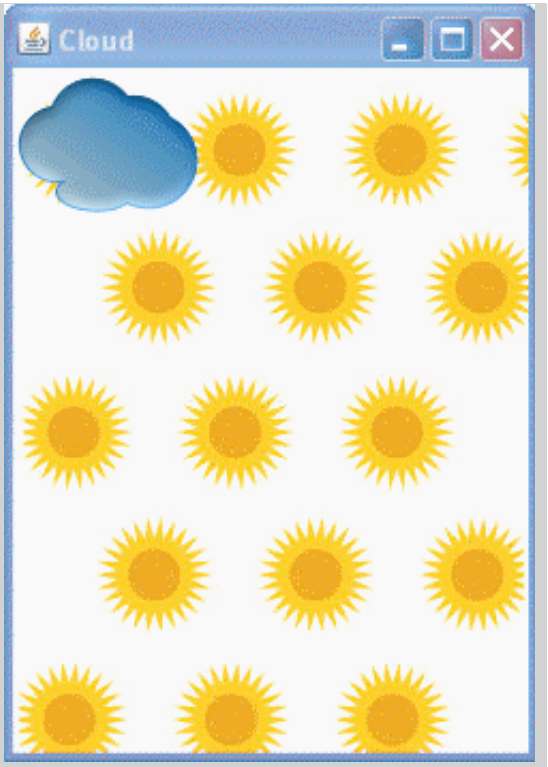

Figure 8: Cloud bouncing within window

Conclusion

This lesson described how to create an animated object and examined interpolated animation. Try the concepts and techniques mentioned in this lesson to explore the other animation capabilities of JavaFX SDK.

« Previous 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | Next »

http://java.sun.com/javafx/1/tutorials/ui/events/ Feb 10, 2009

Building GUI Applications With JavaFX
Lesson 8: Bringing Interactivity to GUI Elements

Download tutorial

Are you ready to bring some interactivity to your application? This lesson shows you how to add behavior to an application by following a step-by-step procedure. Once you understand the concept you can proceed to create more sophisticated interactive applications.

JavaFX Script enables you to make a desktop or a mobile application respond in a pre-programmed way to events through a convenient event-handling mechanism. Each JavaFX object that can potentially expose behavior has instance variables that map to event-related functions. You can define these functions to handle events such as the click of a mouse button, tap of a stylus, or the release of a key. For example, you can define a function that will render text when you click a circle with your mouse. For a complete list of events that can be handled by objects, see the JavaFX Script API.

This lesson is divided into two sections: Common Profile and Desktop Profile. The Common Profile section describes how to handle events in applications that run on mobile devices and on desktop platforms based on an application that draws a circle. The Desktop Profile section describes how to handle events on a desktop. It offers a demo that draws a button and uses rich effects that are specific only to the desktop platform.

Common Profile

This section describes how to handle events in mobile and desktop applications using the common profile API. The application is a circle which changes its fill color depending on events that occur on it. In its original state, the circle is stroked with red and filled with white. When you press the circle it fills with grey; when you click the circle, it fills with orange. If you click the circle again, it changes its fill color back to white. The screen captures of the demo at the described states are shown on the following picture:

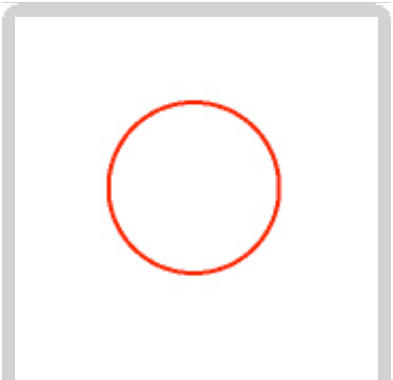

Figure 1: Initial state

The pointer is not on the circle.

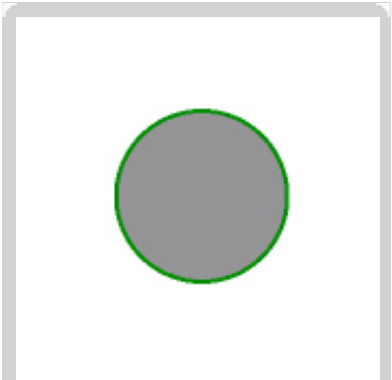

Figure 2: Press Event

You are pressing the circle. The fill color is grey, and the stroke color is green.

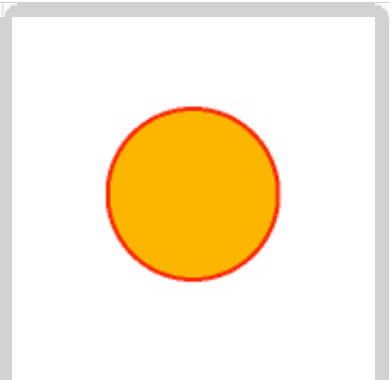

Figure 3: Click event

You released the pointer from the circle. A click occurred, the fill color became orange and the stroke color turned red.

Create a file with an `.fx` extension, for example `FXCircle.fx`. Avoid using file names that match the names of existing classes, instance variables, or reserved words because this leads to errors during compilation. For more information about existing classes, variables, and reserved words, see JavaFX Script API and Learning the JavaFX Script Programming Language.

Adding Graphics

Draw a circle stroked with red and filled with white. The complete code for the graphics is given in the **GraphicsCommon.fx** file.

> Note: If you use this demo on a desktop platform, it is run in a window which is created using the `Stage` class. For the mobile version of the application, the `Stage` class is used to set the scene for the application.

Handling the Click Event

The click event occurs when you click the circle. Depending on the platform you are using, the click event can either be a click of a mouse, a tap of a stylus, or the touch of a finger.

To handle the mouse click event:

1. Define a variable `initState` required for handling the click event.
2. Define the `onMouseClicked` function. The following code shows how to accomplish steps 1 and 2.

```
var initState: Boolean = true;

Stage {
    scene: Scene {
        content:
            Circle {
                ...

                onMouseClicked: function(event){
                    if (initState){
                        colorFill = Color.ORANGE;
                        initState = false
                    }
                    else{
                        colorFill = Color.WHITE;
                        initState = true;
                    }
                }
            }//Circle
    }//Scene
}//Stage
```

As a result, when you click or tap the circle, it changes its fill color from white to orange and vice versa if you click it again.

Handling the Press Event

The press event happens when you press the circle with a stylus or a mouse without releasing it. In this example the press event changes the fill and stroke colors of the circle.

To handle the mouse press event, use the following code:

```
Circle {
    ...
    onMouseClicked: function(event){...}
    onMousePressed: function(event){
        colorFill = Color.GREY;
        colorStroke = Color.GREEN;
    }
}//Circle
```

As a result, the circle changes its fill color to grey and stroke color to green when you press it.

Handling the Release Event

The release event occurs when you release a pointer from the circle. In this example, when you release the pointer from the circle, two events occur - click and release. The stroke color turns to red.

To handle the mouse release event, use the following code:

```
Circle {
    ...
    onMouseClicked: function(event){...}
    onMousePressed: function(event){...}
        onMouseReleased: function(event){
        colorStroke = Color.RED;}
    }
}//Circle
```

In this example, when you release a pointer from the circle, two events occur - click and release. The stroke color turns to red.

Desktop Profile

This section helps you understand how to handle mouse events on the desktop platform by creating an interactive application. The application demonstrates an animated "Play-Pause" button that changes its appearance as you perform various mouse actions. When you enter or exit the button area, or click, press or release the button, the button itself changes its appearance. Several screen captures of the button at each of the described states are shown in the following images.

Figure 4: Mouse is out of the button

Mouse cursor is not on the button, "Play" indicator is displayed, button is not highlighted.

Figure 5: Mouse is on the button

Mouse cursor is on the button area, button is highlighted.

Figure 6: Mouse press

Mouse cursor is pressing the button, but no click has occurred, button is faded.

Figure 7: "Pause" is displayed

Mouse cursor has released the button, click has occurred, indicator has switched from "Play" to "Pause", button is highlighted.

Figure 8: Mouse is out of the button

Mouse cursor has exited the button area, "Pause" indicator is displayed, button is not highlighted.

In the initial state, when the mouse enters the button, the button begins to glow. Then, when you press or click the button, it fades. If you release the mouse button, its indicator changes from "Play" to "Pause". If you click the button again, the "Pause" indicator switches back to "Play".

As this lesson focuses on event handling, it will not describe the actual process for creating the graphics. Instead, the lesson provides the full source code for the graphics and gives detailed steps on how to handle events that occur on those graphics. If you want to know more about creating graphics, refer to Creating Graphical Objects.

Create a file with an `.fx` extension, for example `FXInteractiveButton.fx`. Avoid using file names that match the names of existing classes, instance variables, or reserved words because this leads to errors during compilation. For more information about existing classes, variables, and reserved words, see JavaFX Script API and Learning the JavaFX Script Programming Language.

Adding Graphics

Add graphics to your application. The complete source code for graphics is provided in the **`Graphics.fx`** file.

Note: The circles representing the button, the path for the triangle representing the "Play" indicator, and the path representing the "Pause" indicator are defined in a `Group` because events occur on the graphics that are in this `Group`. For more information about the `Group` class, see JavaFX Script API.

Defining the Cursor

When you create a button, you can define a hand-like cursor that appears when you point to the button.

To define the cursor:

1. Add the `javafx.scene.Cursor` import statement.
2. Specify the `cursor` variable inside the `Group` and set `Cursor.HAND` as its value.

The following code produces the effect:

```
import javafx.scene.Cursor;                    //required to display
                                               //a "Hand" cursor

Stage {
    ...
        scene: Scene {
        content: [
            ...
            Group {
                    ...
```

```
                cursor: Cursor.HAND
                ...
                content: [
                    ...
                ] //Content
            }//Group
        ]//Content
    }//Scene
}//Stage
```

For more information about the `Cursor` class, see JavaFX Script API. For a visual list of cursors in JavaFX SDK, see Quick JavaFX GUI Overview.

Handling the Mouse Enter Event

This event happens when you enter the button area with your mouse cursor. It is controlled by the `onMouseEntered` function.

To handle the mouse enter event define the `onMouseEntered` function.

The event handling functions refer to the `Group` so they should be specified within a `Group`, as shown in the following code:

```
Stage {
    ...
        scene: Scene {
        content: [
            ...
            Group {
                ...
                content: [
                    ...
                ]
                onMouseEntered: function(event){
                    effect.level = 0.65;
                }
            }//Group
        ]//Content
    }//Scene
}//Stage
```

Once you enter the button with the mouse, the button begins to glow. The glowing effect is controlled by the `effect` and `level` variables. The level of glowing is set to `0.65`, which is high enough to see that the cursor is on the button. For more information about the `onMouseEntered` function, see JavaFX Script API. For more information about the `MouseEvent` class, see JavaFX Script API.

Handling the Mouse Click Event

The mouse click event controls the behavior of the button when you click it. It is handled with the help of the `onMouseClicked` function.

To handle the mouse click event:

1. Add the `javafx.scene.input.MouseButton` import. This step will enable handling only left mouse button clicks.
2. Add the `javafx.scene.shape.Shape` and the `javafx.scene.shape.DelegateShape` imports. This will enable morphing "Play" into "Pause".

3. Define variables that are responsible for handling mouse click and mouse press events. Here's the code for steps 1, 2, and 3:

```
import javafx.scene.input.MouseButton;            //required to handle
                                                  //only left mouse button clicks
import javafx.scene.shape.Shape;                  //required to switch
                                                  //between the "Play"
                                                  //and "Pause" indicator
import javafx.scene.shape.DelegateShape;          //required to switch
                                                  //between the "Play"
                                                  //and "Pause" indicator
var start: Boolean = true;                        //required to switch
                                                  //between the "Play"
                                                  //and "Pause" indicator
var currentShape: Shape = startingShape;          //required to define
                                                  //the initial state
                                                  //of the "Play" and "Pause"
                                                  //indicator
var delegate = DelegateShape {                    //required to transform
                                                  //"Play" to "Pause" and
                                                  //vice versa
    shape: bind currentShape
}

Stage {
    ...
}
```

and

```
Stage {
    ...
    content: [
        ...
        Group {
            ...
            content: [
                ...
                    delegate,
            ]//Content
        }//Group
    ]//Content
}//Stage
```

Note: The variables `startingShape` and `endingShape` define the graphics for the "Play" and for the "Pause" indicators correspondingly. These variables are defined in the **`Graphics.fx`** file. The variable `delegate` which is of the `DelegateShape` type defines an object that represents two states of the indicator: "Play" and "Pause". Initially, the value for `delegate` is bound to `currentShape` which is `startingShape`. This means that the `Play` indicator is displayed. For more information about the `DelegateShape`class, see the JavaFX API.

---

4. Add the `javafx.animation.Timeline` import and the `animation` variable. This will enable you to animate the indicator.

```
import javafx.animation.Timeline;                          //required to render animat
```

```
var animation = Timeline{
    keyFrames : [
        at (0s)  {currentShape => startingShape},
        at (1ms) {currentShape => endingShape},
    ]
};
```

For more information about animation, see Creating Animated Objects.

5. Define the `onMouseClicked` function.

```
Group {
    ...
    content: [
        ...
    ]
    onMouseEntered{...}
    onMouseClicked: function(event) {
        if(event == MouseButton.PRIMARY) {
            if (start){
                animation.rate = 1.0;
                animation.play();
                start = false
            }
            else{
                animation.rate = -1.0;
                animation.play();
                start = true
            }
        }
    }//onMouseClicked
}//Group
```

The code samples from this section do the following.

- When you click the button, animation starts. During the animation the `currentShape` transforms into the `endingShape` and switches the indicator from "Play" to "Pause". The transformation happens using the `animation.play` function and the `delegate` object. The `shape` variable of the object is bound to the `currentShape` and this enables it to change depending on the current state of the indicator.
- In the "Pause" state, when you click the button, the animation starts but this time in the opposite direction. The direction of the animation is controlled by the `rate` variable. The indicator switches back to "Play".

Note: These operations are performed only with the left mouse button. This is controlled by the `if(event.button == MouseButton.PRIMARY)` statement.

For more information about the `onMouseClicked` function, see JavaFX Script API. For more information about the `MouseButton` class, see JavaFX Script API.

Handling the Mouse Press Event

This event defines the action that your application takes when the mouse button is pressed. The event is handled using the `onMousePressed` function.

To handle the event, define the `onMousePressed` function as the following code shows:

```
Group {
    ...
    content: [
        ...
    ]
    onMouseEntered {...}
    onMouseClicked {...}
    onMousePressed: function(event){
        if(event.button == MouseButton.PRIMARY) {
            effect.level = 0.0;
            }
    }//onMousePressed
}//Group
```

Once the left mouse button is pressed, the on screen button becomes darker, simulating a push effect. As long as the mouse is pressed, you haven't clicked it, so there is no change in the "Play-Pause" indicator.

Note: Mouse press action is performed only with the left mouse button. This is controlled by the `if(event.button == MouseButton.PRIMARY)` statement.

For more information about the `onMousePressed` function, see JavaFX Script API.

Handling the Mouse Release Event

This event defines what your application does when the left mouse button is released. This action is controlled by the `onMouseReleased` function.

To handle the mouse release event, use the following code:

```
Group {
    ...
    content: [
        ...
    ]
    onMouseEntered {...}
    onMouseClicked {...}
    onMousePressed {...}
    onMouseReleased: function(event) {
        effect.level = 0.65;
    }
}//Group
```

When you release the mouse button, two events happen `onMouseReleased` and `onMouseClicked`: the graphic button in your application remains highlighted as the mouse cursor is still on your graphic button, and the "Play-Pause" indicator changes.

For more information about the `onMouseReleased` function, see JavaFX Script API.

Handling the Mouse Exit Event

This type of event occurs when the mouse cursor exits the button area. The event is defined by the `onMouseExited` function.

To define the mouse exit event, use the following code:

```
Group {
    ...
    content: [
        ...
    ]
    onMouseEntered {...}
    onMouseClicked {...}
    onMousePressed {...}
    onMouseReleased {...}
    onMouseExited: function(event){
        effect.level = 0.3;
    }
}//Group
```

When the mouse cursor exits the area defined by the graphic button, the button's appearance is no longer highlighted. The button returns to its initial state with the glow level at `0.3`. This is the default level for the Glow effect which is set to the Group as the initial highlighting state by the `effect: effect` statement.

For more information about the `onMouseExited` function, see JavaFX Script API. For more information about binding, see Applying Data Binding to UI Objects.

Here is the complete **`FXInteractiveButton.fx`** file.

Conclusion

The JavaFX SDK provides a full range of built-in functions that handle events both in mobile and desktop applications. This example shows how easily some of them can be used to make your application respond to events generated by user actions.